

pyqecclang: An Embedded Quantum Programming Language for Scientific Computing

Author One and Author Two

Affiliation to be determined, City, Country

Abstract

Quantum algorithms for scientific computing—quantum linear-system solvers, Hamiltonian simulation, and the rapidly growing family of quantum ordinary- and partial-differential-equation (ODE/PDE) solvers—have a structure that mainstream quantum programming languages do not support well. These algorithms are stated relative to *mathematical access models* (block-encodings, sparse-access oracles, quantum random-access memories) whose physical realizations are deliberately left open; they compose deep hierarchies of subroutines whose adjoint and controlled versions must remain available; and they carry precision parameters and structural promises that must travel with the code that produced them. We present pyqecclang, a Python-embedded language that addresses these needs through three coupled mechanisms: (i) a *pluggable oracle* design in which abstract oracles are declared with one of nine named paradigms, carried symbolically as open modules in a register-level intermediate representation (RIR), and linked to concrete implementations—gate networks, QRAM resources, or alternative data-loading strategies—by a separate binding pass; (ii) *algorithm protocols*, structural interfaces through which algorithm libraries declare, check, and compose their input models without extending the core language or its IR; and (iii) a library of *quantum scientific computing primitives*—block-encoding algebra, qubitization/QSVT with a self-verifying quantum signal processing phase engine, quantum data loading, reversible fixed-point arithmetic, and a compiler from pure Python numerical functions to reversible circuits—out of which six distinct ODE solver families and several nonlinear PDE paths are assembled. The same open program can be partially bound, re-bound to different implementations, exported to a modular textual IR, executed on native register-level backends—including a new RIR-native interpreter that ships with PySparQ—or checked by a dependency-free reference simulator. We survey existing quantum programming languages and frameworks, derive language requirements from the structure of quantum scientific computing itself, and compare feature and algorithm coverage in detail. We verify the entire library numerically on real backends: a campaign of 473 experiments in thirteen groups executes every algorithm family through up to four independently implemented paths—among them an RIR-native interpreter now shipped with PySparQ and a `Circuit.to_matrix` unitary export contributed to UnifiedQuantum, both created for this campaign—and certifies implementations against truncated-oracle references at register-exact scale, with implementation and method errors reported separately. The campaign surfaced and recorded two genuine defects (an oblivious-amplification iteration and a Schrödingerization momentum sign), which we report openly alongside the certified results; parameter-scaling studies, the repair of the recorded defects, and native support for resource estimation remain future work.

1 Introduction

Quantum algorithms for scientific computing have left the era of isolated prototypes. Since the quantum linear-system algorithm of Harrow, Hassidim, and Lloyd [1], a mature body of work has unified linear algebra under block-encodings and the quantum singular-value transformation (QSVT) [2–4], turned Hamiltonian simulation into a family of interchangeable black-box kernels [5–7], and produced a rapidly growing zoo of quantum solvers for ordinary and partial differential equations: linear-combination-of-Hamiltonian-simulations (LCHS) methods [8], Schrödingerization [9], contour-based matrix-function decompositions, Carleman linearization for nonlinear systems [10–13], quantum finite-element and spectral methods [14, 15], and applications to fluid dynamics [16].

The programming support available for expressing these algorithms has not kept pace. Today, a researcher who wishes to implement, compare, or compose such algorithms faces a mismatch between the *mathematical* structure of the algorithms and the *computational* structure of the tools:

1. **Algorithms are stated relative to access models, not circuits.** QLSS papers assume “given a block-encoding of A ”; ODE solvers assume “given a block-encoding of the generator G and a state-preparation oracle for the initial vector”; input models are specified by sparse-access oracles or quantum RAM (QRAM) with precise XOR semantics [17, 18]. The concrete realization of these oracles—gate-based QROM, a physical QRAM, alias sampling, or an application-specific reversible circuit—is a separate decision that the algorithm text deliberately defers. Mainstream quantum SDKs force the opposite order: an oracle must be a concrete circuit object before the algorithm can be written around it, and switching realization later means rewriting the algorithm.
2. **Programs are deep, symbolic, and large.** Scientific-computing programs assemble solvers from primitives through many layers (input model $\rightarrow$ block-encoding $\rightarrow$ transform $\rightarrow$ solver $\rightarrow$ readout), with iteration counts and system sizes that make eager flattening to gate lists meaningless: a repetition count of 2^{40} is a legitimate parameter in this regime. What is needed is an intermediate representation that *preserves* modular calls and static repetition, so that programs remain small, serializable, reviewable, and composable, and so that backends—not frontends—decide how far to expand.
3. **Correctness obligations are quantitative and compositional.** Block-encoding scale factors α , adjoint and controlled variants of every subroutine, alias-freedom of operands, spectral-norm promises, dissipativity assumptions, and discretization caveats must all be tracked through composition. Languages that model a quantum program as a flat list of gates have nowhere to attach this information; languages that track types but not these domain obligations lose them silently.

This paper describes pyqecclang, a Python-embedded quantum programming language in the embedded-domain-specific-language tradition [19, 20] designed for this workload. pyqecclang (currently at version 0.8.0) takes the position that a quantum scientific-computing language is a *composition and bookkeeping discipline*, not a gate constructor. Its design is organized around three mechanisms:

Pluggable oracles (Section 5). Algorithms are written against *abstract oracles* declared with one of nine named paradigms (unitary, block-encoding, XOR database,

state-preparation isometry, sparse access, phase oracle, reversible function, algorithm stage). Abstract oracles are carried as *open modules*—modules with a signature but no body—in a register-level intermediate representation (RIR). A separate linking pass, `bind`, attaches concrete implementations, checking paradigm, register-type, scale-factor, and capability compatibility, and supports partial binding so that one open program can be linked against gate networks, QRAM resources, or application-specific circuits in any combination. QRAM is a first-class typed resource, and alternative data-loading realizations of the same oracle slot are interchangeable.

Algorithm protocols (Section 5, Section 7). Algorithm libraries declare their input models as Python `Protocol` interfaces (structural types) and expose solver families as small *generator protocols* that map problem objects to quantum programs. Because protocols live in the library, not in the language core, new algorithms—including entirely new solver families—can be added without touching the grammar, the IR, or the serializer, while contract checking (with structured, machine-readable issue reports) remains uniform.

Scientific-computing primitives (Section 6). A library of block-encoding algebra (products, sums, linear combinations, tensor and Kronecker structures), PREPARE–SELECT decompositions with gate, QRAM, and alias-sampling realizations, a qubitization/QSVT/QSP stack with a self-verifying quantum-signal-processing phase engine, reversible fixed-point arithmetic with status flags, and a compiler from pure Python numerical functions (via their abstract syntax trees) to reversible modules. On top of these, pyqecclang assembles *six distinct ODE solver families* (LCHS, Schrödingerization, contour-based matrix decomposition, Carleman linearization for nonlinear ODEs, implicit-Euler history systems solved by quantum linear-system solvers, and truncated Taylor kernels) and several PDE paths, including heat evolution, nonlinear polynomial PDEs via a homotopy-analysis linearization (QHAM), Fokker–Planck dynamics, and a quantum finite-volume method (QFVM) for the compressible Euler equations built from reversible Roe-flux arithmetic (Section 7, Section 8).

Underneath, the register-level IR is the architectural center (Section 4): programs are directed acyclic graphs of *modules* with typed quantum registers and resources; `Call`, `Repeat`, `Control`, and `Adjoint` nodes remain symbolic through serialization and export; adjoint and controlled usage are checked against compositionally inferred capabilities before any backend is invoked; and the IR is deliberately unitary—measurement, reset, and classical feedback are host-side concerns, planned as readout actions rather than IR instructions. The same RIR program can be validated while open, bound and exported to a modular textual backend (OriginIR-ext), lowered to a strict gate set, executed by a dependency-free reference simulator, or executed on native register-level backends—including, since this work, a native RIR interpreter shipped with PySparQ that expands the module graph at interpretation time.

Contributions. This paper makes the following contributions:

- **A requirements analysis** that derives, from the structure of quantum scientific computing itself, eight concrete language requirements (deferred oracle realization, realization independence, structure preservation, capability tracking, symbolic scale, promise provenance, readout separation, and library extensibility), and identifies how existing languages fail them (Section 2, Section 3).

- **A language design** meeting these requirements: an embedded builder over typed register views; a module-preserving register-level IR with an open-module oracle system, a linking pass, capability checking, and a canonical JSON serialization with schema (Section 4, Section 5).
- **A protocol-based algorithm architecture** in which input models, adapters, and solver generators are first-class Python objects with structural contracts and machine-readable reports, and in which the same problem object is interchangeable between solver families and QLSS kernels (Section 5, Section 7).
- **A primitive and solver inventory** for quantum scientific computing, with detailed feature- and coverage-comparison tables against existing languages and frameworks (Section 6, Section 7, Tables 2 and 3).
- **A numerical verification campaign on real backends** (Section 9): 473 experiments in thirteen groups cover every algorithm family, each checked against a correctness oracle that is independent of the implementation, most of them evaluating the entire input domain in a single superposed execution. The campaign runs on infrastructure contributed with this paper—an RIR-native PySparQ interpreter and UnifiedQuantum’s `Circuit.to_matrix`—alongside the pre-existing reference executor and adapter, cross-checks up to four independent implementations, separates implementation from method error, and surfaced two genuine defects, which are recorded openly with reproductions. What remains open is the parameter-scaling axis, the defect repairs, and resource estimation—tracked explicitly rather than overclaimed.

Everything described in this paper is implemented in the open-source pyqecclang package; all code listings are drawn from its doctested tutorials, examples, and test suite.

Outline. Section 2 develops the requirements analysis. Section 3 surveys related languages and frameworks. Section 4 presents the language surface and the RIR. Section 5 presents the oracle and protocol system. Section 6 presents the scientific-computing primitives. Section 7 presents the differential-equation stack, and Section 8 three case studies. Section 9 describes implementation and verification status: the evidence ladder, the numerical-verification campaign with its per-family outcomes and recorded defects, and the deliberately reserved parts (validation at scale and resource estimation). Section 10 discusses limitations and future work.

2 Quantum scientific computing as a language-design problem

The design of pyqecclang did not start from a general-purpose quantum language and move toward applications; it started from the structure of quantum scientific-computing algorithms and asked what a language must provide for that structure to be expressed directly. This section makes that derivation explicit. We first describe the canonical shape of a quantum scientific-computing program (Section 2.1) and then extract eight requirements (Section 2.2), each of which will reappear as a concrete language mechanism in Section 4 through Section 7.

2.1 The shape of a quantum scientific-computing program

Across the literature on quantum linear algebra and differential equations, programs have a remarkably stable layered shape, illustrated in Fig. 1:

1. **Mathematical problem.** A linear system $Ax = b$; an initial-value problem $u' = Gu$, $u(0) = u_0$; a nonlinear polynomial system $u' = \sum_p F_p u^{\otimes p}$; or a PDE before spatial discretization.
2. **Input model (oracle paradigm).** The problem is accessed through a declared access model: a block-encoding of A or G with scale factor α [2]; sparse-access oracles that locate and return nonzero entries; a QRAM serving an XOR database $|a\rangle|d\rangle \mapsto |a\rangle|d \oplus M[a]\rangle$ [17]; or a state-preparation oracle for b or u_0 . Algorithm analyses are always relative to this model, and the model’s parameters (α , sparsity, width) enter the complexity bound [18].
3. **Primitive algebra.** From the input model, larger objects are built: linear combinations of unitaries (LCU) with PREPARE–SELECT pairs [5, 7], tensor products, Kronecker sums, dilations of rectangular or non-Hermitian matrices, projectors, and shifted lattices arising from finite-difference stencils.
4. **Quantum transforms.** Qubitization and QSVT [2, 7] consume block-encodings to apply polynomial transformations of the encoded operator: matrix inversion filters for QLSS, Jacobi–Anger branches for Hamiltonian simulation, eigenstate filters, fixed-point search reflections. Alternatively, product formulas or truncated-series LCUs realize the same functions with different cost/precision trade-offs.
5. **Solver assembly.** Transforms are wrapped into solver skeletons: phase estimation readouts, amplitude-amplification boosts, or, in the ODE/PDE setting, solver families that reduce evolution to Hamiltonian-simulation calls—LCHS and its weighted quadrature of Hermitian branches [8], Schrödingerization’s Fourier-lift embedding [9], contour decompositions, Carleman lifts of nonlinear systems into block-triangular linear ones [12], or implicit-Euler history systems solved by a QLSS in one shot [11, 21].
6. **Readout and norm recovery.** Finally, since the solution is a quantum state, the program ends with a *host-side* plan: phase or amplitude estimation, sample-based norm recovery through an independent matrix-norm probe, classical post-processing such as continued fractions, and statistics over repetitions. Measurement is not part of the unitary algorithm.

Two features of this shape drive everything that follows. First, the interfaces *between* layers are mathematical contracts, and the objects that realize a given contract are interchangeable: the same block-encoding slot may be filled by a gate network, a QRAM lookup, an alias-sampling construction, or an application-specific reversible circuit, with different resource profiles but identical semantics at the contract level. Second, the shape is *deep and parameterized*: a PDE discretization parameter n enters as register widths and repetition counts, not as a longer gate list, and the scientifically meaningful object is the symbolic composition, not any particular expansion.

2.2 Eight requirements

From this structure we extract eight requirements. Throughout, Table 1 summarizes the requirement, its mathematical source, the deficiency it exposes in common practice, and the mechanism that pyqecclang builds for it.

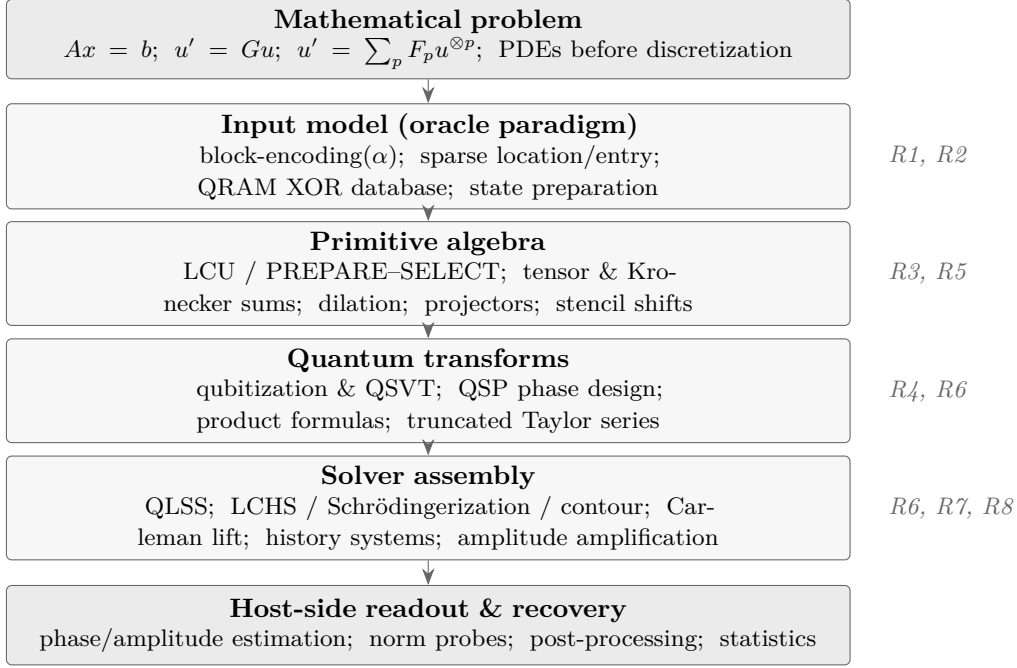


Figure 1: The canonical layering of a quantum scientific-computing program. Each layer is accessed through the layer below it by a well-defined interface; the right margin annotates which of the requirements R1–R8 of Section 2.2 the layer stresses. The top and bottom layers (shaded) are host-side concerns; the language must connect to them without embedding them in the quantum IR.

R1: Deferred oracle realization. Algorithms are published relative to abstract oracles; implementations are chosen later, and the choice is a deployment decision (gate synthesis vs. hardware QRAM vs. native operator), not an algorithmic one. A language must therefore let a program be *complete as an algorithm* while its oracle slots are empty—validatable in that state, shareable in that state, and closable without rewriting. Circuit-construction APIs that require a concrete gate-level oracle object invert this order and make algorithm-first development impossible; systems in which abstraction means “call an opaque host function” lose serializability and reviewability of the open program.

R2: Realization independence. For a given oracle slot, several concrete realizations with different resource/cost profiles must be interchangeable behind one semantic contract, including QRAM-backed ones (where the data table itself is a runtime resource, not part of the program text). This is what makes an algorithm library reusable across input regimes and what makes cost studies—switching one slot from QROM to alias sampling, say—into a one-line change.

R3: Compositional structure preservation. Subroutine calls, static repetition, control, and adjoint blocks must survive end-to-end: in the IR, in its serialization, and in exports. Eager flattening destroys the object to which precision annotations, resource accounts, and review attention attach; it also makes program size explode with scale parameters long before execution is ever attempted. The IR, not the frontend, is the artifact of record.

R4: Adjoint and control as tracked capabilities. Scientific-computing programs use $U^\dagger$ and controlled- U pervasively (uncompute-and-prepare patterns, oblivious amplitude

amplification, eigenstate filtering). Whether a composed object admits adjoint or control is a compositional property of its parts; a language must infer and check it *at composition time*, before any backend is asked to realize it.

R5: Symbolic scale. Parameters such as $\text{Repeat}(2^{40})$ or a 64-qubit addressing width are legitimate. The frontend must never expand them; cost models, not materialization, must guard execution; and even textual export must remain logarithmic in repetition counts. Scale invariants must be part of the specification, not folklore: “serialization must not unconditionally replicate a repeated body” is a testable statement.

R6: Promise and precision provenance. Block-encoding factors α , spectral-norm promises ($\|A\| \leq a$, $\sigma_{\min}(A) \geq \sigma$), dissipativity of the ODE generator, degree choices of polynomial approximations, and known approximation caveats (finite quadrature, omitted contour poles, Carleman tails) are all part of a program’s meaning. None of them is a gate; none of them is safely left in a paper’s appendix. A language must carry them as first-class metadata attached to the objects they qualify, with explicit provenance, and must not pretend they are guarantees: a promise declared without evidence is recorded as a declaration. Conversely, the language core need not, and should not, reify a precision target ε as a type: precision is produced by generator functions and applications, and the language’s job is to keep its provenance auditable.

R7: Readout separation. The unitary algorithm and the measurement plan are different objects with different lifetimes. Embedding measurement in the IR forces every composition boundary to become a host round-trip and prevents coherent reuse of solver outputs. Measurement, reset, post-selection, and statistics belong to a host-side plan that references, but does not contaminate, the IR.

R8: Library extensibility without core changes. The algorithm zoo grows by research, not by committee: LCHS, Schrödingerization, and Carleman-based pipelines were research artifacts within recent memory. A language in which adding a solver family means modifying the grammar, the IR, the serializer, or a central type registry will always lag the literature. New algorithms must be expressible as new libraries—new protocols, new input models, new generator functions—over a fixed core.

Resulting programming paradigm. These requirements fix a three-phase programming paradigm that recurs throughout pyqecclang:

Generate programs in Python against declared input models; *store and share* them as RIR—open or bound; *export or execute* on backends, with readout planned at the host.

Within generation, the differential-equation stack (Section 7) adds a sharper discipline that we call *problem–adapter–protocol*: the *problem layer* keeps the mathematical input paradigm native to the application (a PDE, a polynomial ODE, a sparse matrix); *adapters* explicitly construct the objects an algorithm needs (block-encodings, lifted initial states); and *protocols* are Python functions that generate new oracles—whose own implementations may still be open. RIR stores the result: generated modules plus the not-yet-implemented oracle slots they depend on. The division of responsibility is explicit: the language is responsible for composability; precision is the responsibility of generator functions and applications (R6).

Table 1: Requirements derived from the structure of quantum scientific computing, common-practice deficiencies, and the corresponding pyqecclang mechanisms. Each mechanism is detailed in the referenced section.

	Requirement	Deficiency in common practice	pyqecclang mechanism (section)
R1	Deferred oracle realization	Oracle must be a concrete circuit before the algorithm can be written	Open modules: abstract oracles with named paradigms carried as body-less <code>Modules</code> ; validation and serialization work while open (Section 5)
R2	Realization independence	Realization choice hard-wired into algorithm code	One oracle slot, many bindings: gate networks, QRAM-typed resources, alias sampling; partial and repeated binding via the <code>bind</code> linking pass (Section 5)
R3	Structure preservation	Frontends flatten to gate lists; exports lose call structure	Module-preserving RIR: <code>Call/Repeat/Control/Adjoint</code> stay symbolic in IR, JSON, and export (Section 4.3)
R4	Tracked adjoint/control	Adjoint/control availability discovered (or not) at backend	Compositional capability inference checked against contextual usage at validation time (Section 4.4)
R5	Symbolic scale	Counts materialized eagerly; programs explode with system size	<code>Repeat</code> up to $2^{63} - 1$ with logarithmic export; cost-model guards; unrolling forbidden in generation and serialization (Section 4.3)
R6	Promise provenance	α , norm bounds, dissipativity, approximation caveats live in papers, not code	First-class attributes (α , <code>SpectralPromise</code> with evidence strings, dissipativity flags, per-method pending items) attached to modules and contracts (Section 5, Section 7)
R7	Readout separation	Measurement embedded in circuits blocks coherent composition	Purely unitary IR; readout as host-side actions planned after export (Section 5.4)
R8	Extensibility	New solvers require core language changes	Library-level <code>Protocol</code> contracts and generator protocols; new algorithms define new protocols in their own files (Section 5.3)

3 Related work

We situate pyqecclang among quantum programming languages and frameworks, then among algorithm libraries, and finally record the feature comparison of Table 2 and the coverage comparison of Table 3. The discussion is organized by the requirements R1–R8 of Section 2.2; general surveys of the field are available [22–24].

3.1 Quantum programming languages

Functional and typed language lines. Quipper [25] pioneered scalable, circuit-generating functional programming: a Haskell host generates gate-level circuits, with “boxing” providing a form of abstraction over generated subcircuits. Scaffold [26] embedded modules and classical control in a C-like surface, compiled by Scaffold with early resource-estimation passes. Notably, Quipper later served as the vehicle for landmark resource analyses of quantum linear systems—although as external studies over generated

circuits, not as a language mechanism [27, 28]. QWIRE and the Proto-Quipper line developed linear and dependent type disciplines for safety guarantees [29, 30]. Silq [31] introduced safe automatic uncomputation through its type system—a genuine language-level answer to the invertibility half of our R4. None of these lines, however, provides deferred-oracle semantics with paradigm contracts and late linking (R1/R2): abstractions expand to circuits, and the open state of a program is not a shareable artifact. Their IRs target gate-level compilation rather than module-preserving program storage (R3/R5), and none ships a scientific-computing algorithm stack.

Industrial SDKs and embedded frameworks. ProjectQ [32] established the Python-embedded pattern, shipped one of the earliest resource estimators, and—through its extensible compiler-engine chain—explicitly supports *emulating* quantum programs at a high level of abstraction, mimicking large oracles instead of compiling them to gates: a genuine form of deferred oracle at execution time, though one that lives in the engine stack rather than in a serializable open program. Quil and pyQuil [33] and Cirq [34] provide circuit DSLs centered on NISQ execution. Qiskit [35] grew a large circuit library and textbook algorithm implementations; demonstration-level HHL shipped historically in its algorithms module and has since been retired with it, and no QSVT or block-encoding abstraction exists in the library today. Its objects are concrete circuits, its transpiler flattens structure, and alternative oracle realizations are separate programs rather than bindings of one open slot. OpenQASM 3 [36] standardizes a textual interchange language whose subroutines and loops preserve some structure, and whose “opaque” declarations gesture toward R1—but at gate granularity, without paradigm contracts, capabilities, or a linking discipline. Closest to our primitive layer among embedded frameworks, Qrisp [37] automates uncomputation and resource-friendly scheduling inside Python and exposes a `BlockEncoding` class—block-encodings as programming abstractions with a NumPy-like interface—which we regard as independent evidence that scientific-computing primitives belong at the language level. Qrisp does not, however, treat the open program as an artifact: there is no paradigm-contracted oracle declaration with late, partial, repeatable linking, no module-preserving serialized IR with promise provenance, and no ODE/PDE solver families. `t|ket>` [38] and `staq` [39] are compiler and synthesis toolchains, complementary to our concerns: they operate *below* the register/module level our RIR preserves.

Q#. Q# [40] is the closest prior art at the language level. Its operations with `Adjoint/Controlled functors` anticipate our capability discipline (R4): specialization availability is declared and tracked. Passing operations as parameters—the closest mainstream analogue to our oracle slots—lets algorithms be written against any conforming implementation, so Q# partially meets R1 at *call* time. Its standard libraries include fixed-point numerics and estimation primitives, and the surrounding Azure tooling ships a serious resource estimator. What Q# does not provide for our workload is the *open program*: oracles are operations with concrete bodies; “partial application” is host-level; there is no serialized artifact in which an algorithm carries its unimplemented input models with paradigm, α , capabilities, and provenance (R1/R2/R6); and its libraries do not include block-encoding algebra, a QSVT/QSP engine, or any ODE/PDE solver families (Table 3). The distinction is one of *when* semantics attach: Q# resolves capabilities at compile time of concrete operations, whereas pyqecclang keeps the *link* itself as a first-class, repeatable, partial operation on stored programs.

Table 2: Language and framework comparison against the requirements of Section 2.2. ✓ = provided; ⊙ = partial (see text); – = absent. Claims are referenced to the systems’ publications and documentation.

	Quipper	Scaffold	Q#	ProjectQ	Qiskit	pyqecclang
R1 deferred oracles	– ^a	–	⊙ ^b	⊙ ^c	–	✓
R2 realization independence	–	–	⊙	– ^c	⊙	✓
R3 structure-preserving IR	⊙	⊙	⊙	–	–	✓
R4 adjoint/control tracked	⊙	–	✓	–	–	✓
R5 symbolic scale ($2^{40}+$)	⊙	⊙	⊙	–	–	✓
R6 promise provenance	–	–	–	–	–	✓
R7 readout separation	⊙	⊙	–	–	–	✓
R8 library extensibility	⊙	⊙	⊙	⊙	⊙	✓
Scientific primitives ^d	–	–	⊙	–	⊙	✓
ODE/PDE solver families	–	–	–	–	–	✓
Resource estimation	⊙ ^e	✓	✓	✓	⊙	– ^f

^aBoxes abstract over generated subcircuits, but there is no open-program state. ^bOperations are passed as parameters and remain abstract to the algorithm, but bodies are concrete and no link-time openness or paradigm contracts exist. ^cThe extensible engine chain can emulate oracles at execution time, but not in a serializable program artifact. ^dBlock-encoding algebra, QSVT/QSP engine, sparse-access and QRAM input models, reversible fixed-point arithmetic with a math-function compiler. ^eScoped and reserved (Section 9.4); the IR preserves the structure an estimator needs.

Research codebases for scientific algorithms. Individual algorithm implementations—LCHS, Schrödingerization, Carleman pipelines, QLSS prototypes—exist as research code accompanying publications [8, 8, 9, 12, 13]. Each typically fixes one method, one input model, and one small simulator; none offers shared input-model objects, interchangeable kernels, or a common IR. The absence of a unifying language layer is precisely the gap this paper documents: the algorithms compose mathematically (all consume block-encodings; all produce state oracles) but not computationally, absent exactly the contracts of Section 5.

3.2 Positioning

The comparison is not a claim that prior systems are poorly built; they are built for a different workload. Circuit SDKs optimize for NISQ execution and hardware co-design; typed language lines optimize for semantic safety of full quantum computation; Q# optimizes for a vertically integrated development kit. pyqecclang optimizes for the fault-tolerant scientific-computing workload, where the unit of progress is a *mathematical algorithm against declared access models*, the unit of sharing is a *program artifact with open slots and recorded promises*, and the unit of validation is *graduated evidence* rather than a single simulator’s output. To our knowledge, no previously published system combines these

Table 3: Algorithm and solver coverage: pyqecclang against the major frameworks’ standard libraries (research-prototype implementations published outside a framework’s libraries do not count). ✓ = provided as a composable, contract-checked library component; ⊙ = demonstration-level implementation; – = absent.

Capability	Q#	ProjectQ	Qiskit	Cirq	pyqecclang
Query/search/amplification	✓	✓	✓	✓	✓
Phase/amplitude estimation	✓	✓	✓	⊙	✓
Hamiltonian simulation (Trotter)	✓	⊙	✓	⊙	✓
Hamiltonian simulation (LCU/Taylor, QSVT)	–	–	⊙	–	✓
Block-encoding algebra	–	–	⊙	–	✓
QSP phase synthesis (self-verifying)	–	–	–	–	✓
Quantum linear systems (QLSS)	–	–	⊙ ^a	–	✓
QRAM/QROM input models (typed resource)	–	–	⊙	–	✓
Reversible fixed-point arithmetic	✓	–	–	–	✓
Math-function compiler (Python → reversible)	–	–	–	–	✓
ODE: LCHS	–	–	–	–	✓
ODE: Schrödingerization	–	–	–	–	✓
ODE: contour decomposition (CBMD)	–	–	–	–	✓
ODE: Carleman (nonlinear)	–	–	–	–	✓
ODE: history systems via QLSS	–	–	–	–	✓
PDE: heat / Fokker–Planck input models	–	–	–	–	✓
PDE: symbolic nonlinear PDEs (QHAM)	–	–	–	–	✓
PDE: Euler equations (QFVM/Roe)	–	–	–	–	✓

^aA demonstration-level HHL circuit shipped historically in Qiskit’s (since-retired) algorithms module; it is not part of the current library.

three units; Tables 2 and 3 make the comparison concrete, and the honest counterpoint is equally explicit: resource estimation—strong in Scaffold, Q# tooling, and ProjectQ—is the deliberately reserved axis of Section 9.4.

4 The language surface and the register-level IR

This section presents pyqecclang from the bottom of the stack upward: the embedding model and register views (Section 4.1), instruction and control structure (Section 4.2), the register-level IR and its invariants (Section 4.3), capability checking (Section 4.4), workspace management (Section 4.5), and the backend-facing architecture (Section 4.6, Fig. 4).

4.1 An embedded builder over typed register views

pyqecclang is embedded in Python (version 3.11 or later) and has no third-party runtime dependencies: the core is a builder library whose output is data, not an executed program. A module is built by declaring typed quantum registers and emitting operations on *views* of them:

```

from pyqecclang import Bits, Builder, export_originir, simulate

b = Builder("bell_pair", {"pair": Bits(2)})
b.h(b["pair"][0])
b.xor(b["pair"][0], b["pair"][1])
program = b.finish().program()

print(simulate(program).amplitudes)          # {(0, 0): 1, (3, 3): 1} /
      ↪ sqrt 2
print(export_originir(program).text)

```

Listing 1: A minimal pyqecclang program (from the doctested tutorial suite). Register index 0 is the least significant bit; fuse concatenates views across root registers; simulate is the dependency-free reference executor.

Registers carry one of four storage kinds—`bits`, `uint`, `sint`, or `rational`, each with a width in $[0, 64]$ —chosen so that downstream register-level backends can map them to native integer registers; total program width is unbounded. A *view* (an internal `Ref`) is a list of *spans* (register, start, width): slices of root registers and fused concatenations across them, possibly *reinterpreted* to a different storage kind. Views are formation-time objects: building, slicing, or reinterpreting a view emits *no* quantum instructions. Consequently the IR never contains physical qubit indices at all; register names, widths, and views survive verbatim to backends (R3).

Builders enforce a strict *alias-freedom ABI*: operands of a single instruction must not overlap, control registers are protected inside controlled blocks, and validation rejects violations before any backend sees the program. Aliasing analyses of this kind are a prerequisite for the uncompute-and-prepare discipline that scientific-computing circuits use pervasively.

4.2 Instructions and control flow

The closed instruction set is deliberately small: broadcast unary gates (`h`, `x`, `y`, `z`, `s`, `t`, `rx`, `ry`, `rz`, `phase`, `global_phase`), reversible bit operations (`xor`, i.e., a CNOT per bit, and `swap`), constant modular addition (`add_const`), and QRAM Load. All richer structure—arithmetic, oracles, entire solver kernels—is expressed as *module calls*, so that the IR’s semantics remain auditable at a fixed size.

Control and repetition are structured blocks, entered as Python context managers:

```

b = Builder("main", {"word": UInt(8)})
with b.repeat(1 << 40):
    b.add_const(b["word"], 1)
p = b.finish().program()

assert len(p.main.body) == 1          # one Repeat node
assert export_originir(p).text.count("DEF ") < 45  # log-time export

```

Listing 2: Symbolic scale: a legitimate program with a repetition count of 2^{40} . The finished program contains a single Repeat node; export remains logarithmic (this is asserted in the test suite).

Repetition counts may range up to $2^{63} - 1$; the body must be structurally valid even at count zero; and—as a normative rule of the IR specification—generation and serialization must never replicate a repeated body according to its count (R5). Guards against runaway expansion are cost models (`expanded_steps` budgets consumed by the lazy expansion iterator), not eager materialization. Control blocks are coherent: control values are compile-time constants attached to protected registers, and nested controls must not

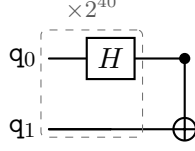


Figure 2: A Repeat block drawn by the quantikz export. The dashed group is a single Repeat node with a static count of 2^{40} ; the body appears exactly once in the program, its serialization, and every export—expansion, if a backend performs it, is logarithmic and leaves no trace in the IR (Listing 2).

Table 4: RIR 0.3 objects and instruction nodes. All nodes are immutable; bodies are tuples. Module with body = $\emptyset$ is an open (oracle) declaration.

Object	Kind	Contents / semantics
Program	container	entry module, module table, IR version
Module	definition	name; registers (quantum interface); resources (QRAM); locals (zero-in/zero-out workspace); attributes (e.g. <code>be_alpha</code> , <code>paradigm</code> , <code>provenance</code>); body or $\emptyset$
RegType	type	kind (<code>bits</code> / <code>uint</code> / <code>sint</code> / <code>rational</code>) and width 0..64; total width unbounded
Ref	view	tuple of spans (register, start, width); declarative, never contiguous physical qubits
QRAM	resource	address width, data width; runtime data binding
Primitive	instruction	broadcast unary gate, <code>xor</code> , <code>swap</code> , <code>add_const</code> , <code>global_phase</code>
Load	instruction	QRAM access $ a\rangle d\rangle \mapsto a\rangle d \oplus M[a]\rangle$; self-inverse
Call	instruction	module invocation; kind-and-width-exact, alias-free argument matching; modules are shared, never copied
Repeat	instruction	static count 0.. $2^{63}-1$; body preserved, never replicated
Control	instruction	coherent conditioning on protected registers with compile-time values
Adjoint	instruction	symbolic inversion; no body rewriting

overlap. **Adjoint** blocks are symbolic: their semantics are defined without rewriting the body, so the adjoint of a composition is available without duplicating it.

4.3 The register-level IR (RIR)

The IR is the architectural center of the language. A program is $\langle \text{entry}, \text{modules}, \text{version} \rangle$; a module is a named, typed interface (ordered quantum registers, ordered QRAM resources, optional private *locals*) plus a body—an immutable tuple of instructions—or *no body at all*, in which case the module is an *open declaration*: an oracle slot (Section 5). Table 4 summarizes the node kinds.

Four invariants define the IR’s contract with the rest of the system:

1. **Modularity end-to-end (R3, R5).** `Call`, `Repeat`, `Control`, and `Adjoint` are preserved in the IR, in its JSON serialization, and in exports. Lowering rules permit backends to expand register operations to physical gates, but module definitions and calls must be preserved, and any expansion happens in the backend without feeding back into the RIR.
2. **Purity.** RIR represents quantum operations after compile-time parameter evaluation—all widths, integers, and angles are concrete. It contains no mea-

surement, reset, runtime classical feedback, or opaque host callbacks (R7); readout is planned host-side (Section 5.4).

3. **Canonical, hardened serialization.** Programs serialize to canonical JSON (sorted keys, deterministic output) under a versioned JSON Schema (currently RIR 0.3, Draft 2020-12). Decoding is strict: unknown tags or fields, duplicate keys, non-finite floats, and unknown versions are rejected. Round-trips are byte-identical, and open modules round-trip unchanged. Readers accept prior format versions, so stored programs are forward-compatible artifacts of record.
4. **Attributes never change instruction semantics.** Domain metadata—block-encoding factors, paradigm tags, implementation-status and provenance strings (R6)—ride along as attributes without affecting the operational meaning of the module they annotate.

Export to the modular textual backend (OriginIR-ext) illustrates the preservation contract concretely. Listing 3 shows the exported text of a QRAM-lookup demo program: module definitions (DEF), calls, and the QRAMDECL resource declaration survive as first-class textual constructs; nothing is inlined.

```

GRAMDECL ram_values 2,3
QINIT 5
CREG 0
DEF m_lookup_2_3_54524a0d1220cf4bd723f2aa(v_address[2], v_data[3])
ram_values v_address[0], v_address[1], v_data[0], v_data[1], v_data[2]
ENDDEF
DEF m_wrapped_lookup_16dc443ca6e4d78bd9ef03a6(v_a[2], v_d[3])
m_lookup_2_3_54524a0d1220cf4bd723f2aa(v_a[0], v_a[1], v_d[0], v_d[1],
  ↪ v_d[2])
ENDDEF
DEF m_demo_b89906731f4fe79582ed7931(v_address[2], v_data[3])
H v_address[0]
H v_address[1]
X v_data[0]
m_wrapped_lookup_16dc443ca6e4d78bd9ef03a6(v_address[0], v_address[1],
  v_data[0], v_data[1], v_data[2])
ENDDEF
m_demo_b89906731f4fe79582ed7931(q[0], q[1], q[2], q[3], q[4])

```

Listing 3: OriginIR-ext export of a nested QRAM-lookup program (verbatim from the repository’s example artifacts). DEF blocks, module calls, and the QRAMDECL resource are preserved; no call is inlined. Content hashes in definition names make exports deterministic and diff-stable.

4.4 Adjoint and control as a capability discipline

Whether an object may be used as $U^\dagger$ or as controlled- U is inferred bottom-up from its parts: a primitive gate admits both; a Call admits adjoint (respectively, control) only if its callee does; open oracle declarations advertise their capabilities as part of the declaration, and the binder checks that the provided implementation actually possesses them (Section 5). Validation then checks every contextual use: wrapping a call in an Adjoint block demands invertibility of the callee *at composition time*. This is a modal capability system in the practical sense—R4—and it eliminates a common class of late-stage backend failures.

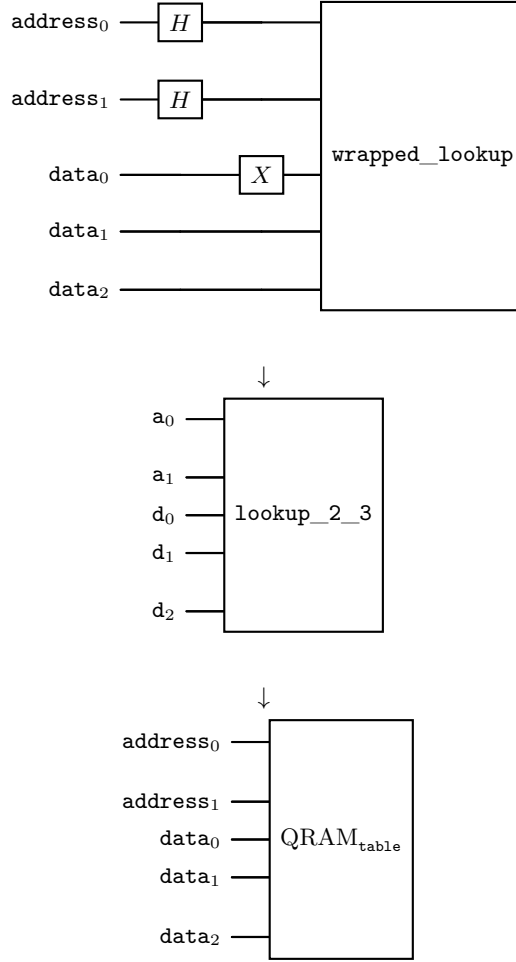


Figure 3: The three modules of Listing 3, drawn independently by the quantikz export of the same program: `demo` (top) prepares address and data and calls `wrapped_lookup` (middle), which calls `lookup_2_3` (bottom), the module that owns the Load on the `table` resource. At every level a call is a box with a name and a typed interface; nothing is expanded, matching the DEF chain of the textual export.

4.5 Workspaces and qubit economy

Modules may declare private `locals`: workspace registers that must be returned to zero at exit (enforced during execution; dirty workspaces raise). A layout pass computes sequential-reuse qubit budgets for module-private locals, so that deeply composed programs remain qubit-frugal without the programmer managing temporary registers by hand. The math-function compiler (Section 6.5) leans on this discipline heavily: its compute / XOR-update / uncompute pattern is only sound because workspace zeroing is an enforced ABI, not a convention.

4.6 Architecture: generation, storage, export, execution

Fig. 4 shows the complete architecture. The separation follows the three-phase paradigm of Section 2: Python generation (builders, algorithm libraries, protocols and contracts), RIR storage (validation, linking, serialization, layout), and backend export or execution. Native backends are imported lazily at execution entry only—generating and serializing

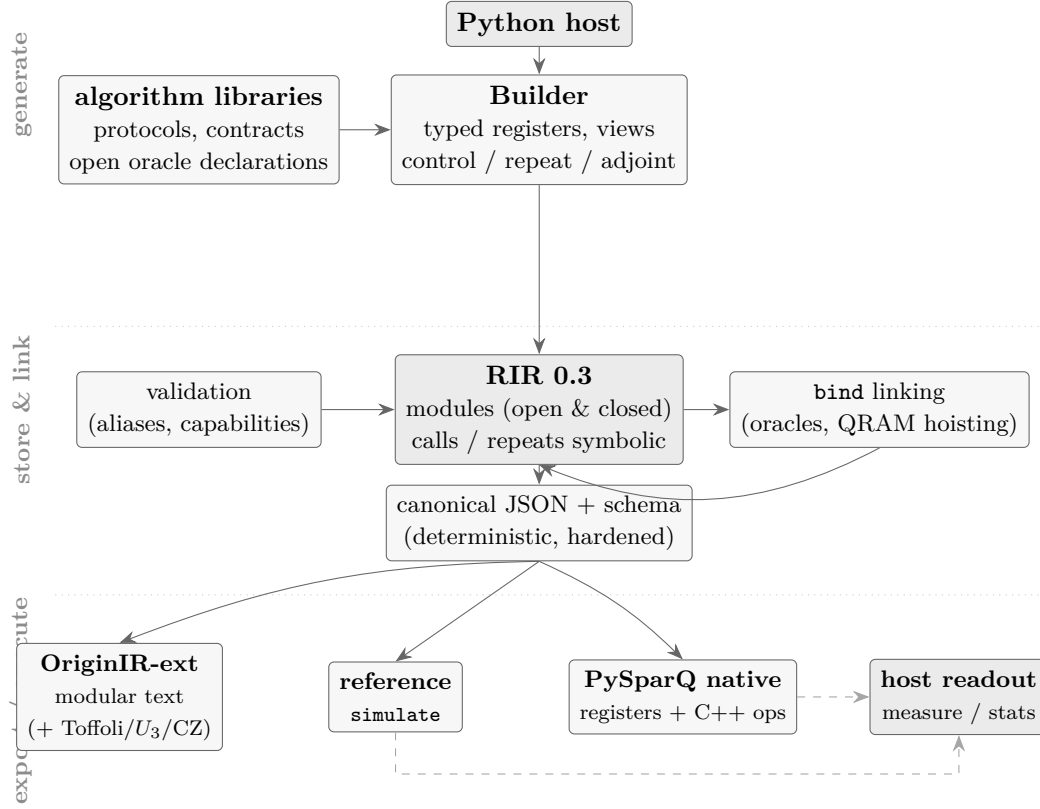


Figure 4: pyqecclang architecture. Programs are generated by the embedded builder and algorithm libraries, stored as RIR (open oracle slots included), optionally linked by `bind`, serialized to canonical schema-validated JSON, and only then exported or executed. Native backends are imported lazily at execution entry; the core has no backend dependency. Readout is a host-side plan, not an IR construct.

RIR requires no simulator installation, and the core has no backend dependency. Backends currently comprise:

- **export_originir**: modular textual export (originally to the OriginIR-ext dialect understood by the *uniquc* simulator), with an additional strict lowering of the whole artifact to the $\{\text{TOFFOLI}, U_3, \text{CZ}\}$ gate set with ancilla pooling;
- **simulate**: a dependency-free sparse-statevector reference executor with step and state budgets, enforcing workspace-zeroing and memory-binding rules;
- **run_pysparq**: register-level native execution on PySparQ, including a registry for *native operators*—module-level implementations (e.g. large reversible arithmetic) compiled on demand to C++ and charged unit cost rather than expanded;
- **quantikz**: circuit-diagram export for documentation and review, preserving the same modularity discipline (calls as named boxes, open slots dashed, structured blocks grouped); every circuit figure in this paper is generated by it.

Measurement, reset, and statistics are planned as host-side *readout actions* appended after export (Section 5.4).

Table 5: The nine oracle paradigms. Each paradigm fixes the mathematical contract of the slot; concrete implementations are interchangeable as long as they honor the contract, the register-type tuple, the scale factor α , and the declared capabilities.

Paradigm	Contract
<code>unitary</code>	a unitary on the declared registers
<code>block_encoding</code>	$(\alpha, a, 0)$ -block-encoding of a target operator: a unitary whose top-left block is A/α ; α is a slot attribute
<code>database_xor</code>	$ a\rangle d\rangle \mapsto a\rangle d \oplus M[a]\rangle$ for a memory M
<code>state_prep_isometry</code>	prepares a target state (or applies an isometry) from a fixed zero input, with a clean-workspace convention
<code>sparse_location_inplace</code>	in-place oracle for the <i>location</i> structure of a sparse matrix access
<code>sparse_entry_xor</code>	XOR oracle returning matrix <i>entries</i> of a sparse access model
<code>phase_oracle</code>	applies a phase function of the input register
<code>reversible_function</code>	a reversible classical function on bit registers
<code>algorithm_stage</code>	an opaque algorithm fragment with declared capabilities (used to stage parts of solvers)

5 Pluggable oracles and algorithm protocols

This section presents the mechanism that addresses R1, R2, R6, and R8: the oracle lifecycle (Section 5.1), QRAM as a first-class resource (Section 5.2), and the protocol/contract architecture (Section 5.3). Fig. 5 summarizes the lifecycle; Listing 4 shows it end-to-end on a small example.

5.1 The oracle lifecycle: declare, carry, inspect, bind

Declare. An abstract oracle is declared by naming it, giving it a typed register signature, and choosing a *paradigm*: one of nine named contracts (Table 5) that fix the mathematical interface the implementation must honor. The declaration optionally carries resources (typed QRAM slots), capability flags (`supports_adjoint`, `supports_controlled`), and attributes. Internally the declaration is a `Module` whose body is absent—an *open module*. Openness is a first-class IR state, not an error state.

Carry. Because an open module is a valid module, an algorithm that calls it is a valid program: validation checks types, alias-freedom, capability demands, and call-graph acyclicity exactly as for closed programs; serialization round-trips the open state; structural tests run unchanged. An algorithm published against an open oracle slot is complete *as an algorithm* (R1), and the IR records precisely which slots remain open and why.

Inspect. Gap analysis (`unresolved`) walks the call graph breadth-first from the entry and reports every reachable open slot with its name, paradigm, shortest call path, register signature, and attributes. Only reachable slots count: an algorithm library that declares several candidate inputs but uses only some of them is not penalized. This makes open programs self-describing artifacts for review, cost analysis, and tooling.

Bind. The linking pass `bind(program, bindings)` closes slots: each slot’s body is rewritten to a single call of the provided implementation, subject to compatibility checks—exact register-type-tuple equality, paradigm match, block-encoding factor α match, and possession of any capabilities the slot advertised. Binding is *partial* (unlisted slots stay

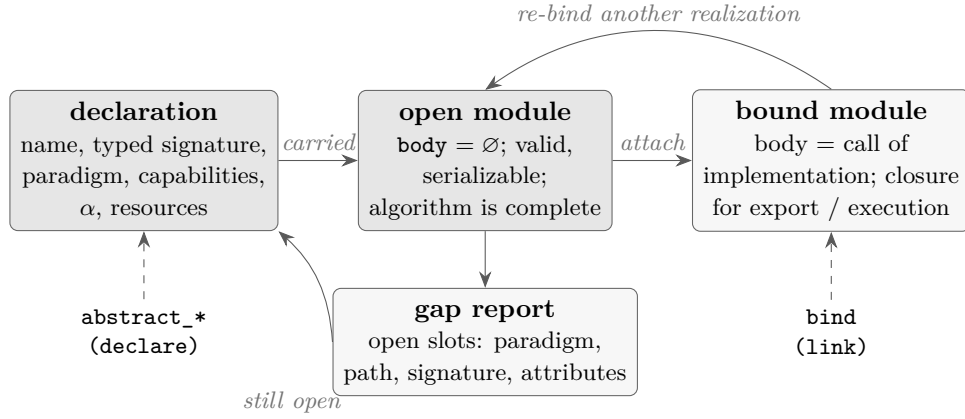


Figure 5: The oracle lifecycle. Declarations create open modules that are carried inside otherwise-complete programs; gap analysis reports what remains open; the binding pass attaches implementations (partially and repeatedly, with compatibility checks and QRAM resource hoisting) until the program is closed for export or execution.

open) and *monotone* (an incompatible binding is rejected without touching the program). When the implementation captures QRAM resources, the linker hoists them to entry-level resources, renames captures to collision-free names along the call graph, and threads the resource bindings through—a classical linker discipline, applied to a quantum IR. Export and execution require closure: `validate(require_closed=True)` lists any slot that would prevent it. The same open program can be bound repeatedly against different implementations (R2): a gate network for a small instance, a QRAM table for a large one, an application circuit for production.

```

# 1. Declare the input, write the algorithm against it.
from pyqecclang.algorithms.oracles import abstract_database
from pyqecclang.algorithms.oracle_algorithms import bernstein_vazirani
from pyqecclang import unresolved

given = abstract_database("BooleanFunction", 3, 1)  # XOR database slot
opened = bernstein_vazirani(given).program()
assert [item.name for item in unresolved(opened)] == ["BooleanFunction"]

# 2. Bind a gate implementation; the program closes and runs.
from pyqecclang.algorithms.oracle_algorithms import
    ↪ affine_boolean_oracle
from pyqecclang import bind, simulate

implementation = affine_boolean_oracle(3, secret=5, bias=1)
closed = bind(opened, {"BooleanFunction": implementation.operation})
assert not unresolved(closed)
state = simulate(closed)
probability = sum(abs(a)**2 for k, a in state.amplitudes.items() if
    ↪ k[0] == 5)
assert abs(probability - 1) < 1e-12

# 3. The same open slot can instead bind a QRAM realization:
#     bind(opened, {"BooleanFunction":
#         ↪ Binding(qram_database(3, 1).operation, {"table":
#             ↪ "truth"}})})
#     with the runtime data table supplied separately.

```

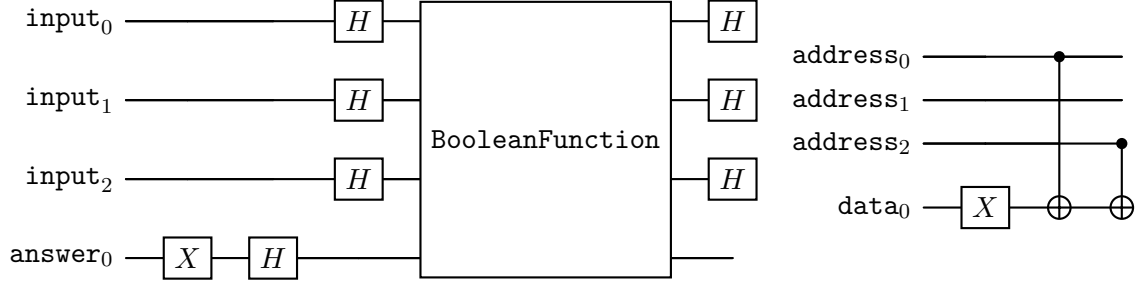


Figure 6: The bound Bernstein–Vazirani program of Listing 4, drawn by the quantikz export. Left: the algorithm as generated—three input wires, one answer wire, and the oracle as a single box named by its slot (`BooleanFunction`); this is the whole program, with no oracle internals materialized. Right: the realization that bind attached for this run (the affine oracle with secret 5, bias 1): an X on the data wire and two XOR controls from address bits 0 and 2. The two circuits are modules of one program; the algorithm circuit is unchanged whichever realization closes the slot.

Listing 4: The oracle lifecycle on Bernstein–Vazirani (verbatim from the doctested tutorial). The algorithm is written and *simulable* only after binding; the same open program later binds a QRAM realization instead of the gate network.

5.2 QRAM as a first-class resource

Data access is where scientific-computing programs differ most sharply from textbook circuits, and where the difference between an addressable memory and an amplitude illusion matters most. `pyqecclang` makes QRAM a *typed resource* of a module: `QRAM(address_width, data_width)` declared alongside registers, consumed by `Load` instructions with precise XOR semantics [17, 41], self-inverse and therefore reversible. The data table itself is *runtime state*, not program text: it is bound at execution (or recorded beside an export), can be patched locally between runs, and never inflates the serialized program (R5). Concrete data-loading families in the library include select-swap QROM with an explicit partition cost model, alias-sampling PREPAREs, banked multi-word QRAM, and gate tables—all interchangeable realizations of the same oracle slots (R2).

One subtlety deserves emphasis because it is a recurring correctness trap in hand-written code: an XOR database of *angles* is not an amplitude oracle. The library’s diagonal block-encoding adapter, for instance, realizes $D_{jj} = \alpha \cos(s d_j/2)$ from an angle table d_j ; feeding a fixed-point table of coefficients c_j and declaring the result `diag( $c_j$ )` would be wrong, and the input model makes the distinction explicit in the slot’s semantics rather than in a comment.

5.3 Algorithm protocols and contracts

R8—new algorithms without core changes—is met by pushing all *algorithmic* typing out of the language core and into the library, using Python’s structural protocols. The mechanism has three layers:

Structural interfaces. The library defines small `typing.Protocol` interfaces (`runtime_checkable`) that any object may satisfy structurally: `UnitaryProtocol`,

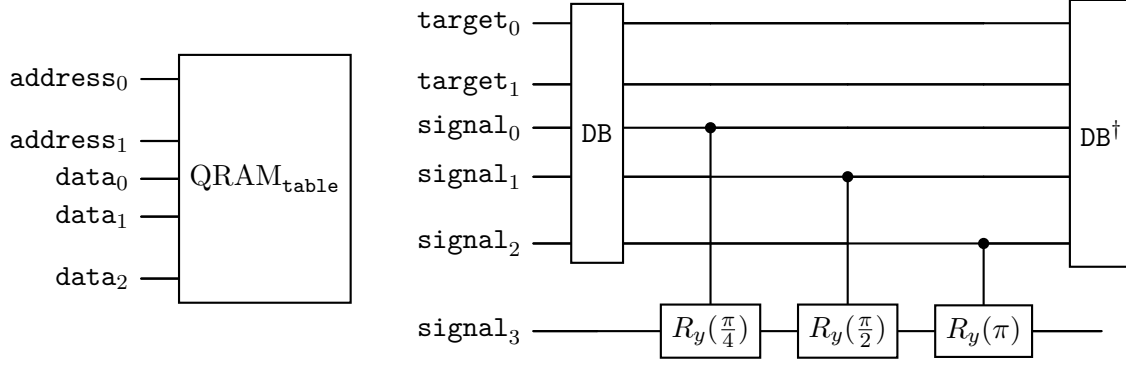


Figure 7: Two QRAM idioms, drawn by the quantikz export. Left: a bare Load—one instruction on the address and data registers, $|a\rangle|d\rangle \mapsto |a\rangle|d \oplus M[a]\rangle$, with the table itself a runtime resource. Right: the diagonal block-encoding of an angle table ($D_{jj} = \alpha \cos(s d_j/2)$, here $s = \pi/4$, $\alpha = 1$): query the angle database, apply one controlled R_y per angle bit onto the flag wire, unquery. The same ladder realizes the coefficient angle databases of the input-model matrix (Table 9); the database box is again a slot, bindable to a gate truth table or a QRAM resource.

BlockEncodingProtocol, StatePreparationProtocol, XorDatabaseProtocol, and StateOracleProtocol, among others. The core Operation object implements all applicable adapters, so a finished builder module is simultaneously a unitary, a state preparation, and a (trivial) block-encoding; algebra such as lcu consumes any objects satisfying the right protocol, whatever their provenance. User applications may define their own protocols—they are ordinary Python classes with no registration step—and check them with `requires(value, Interface)`.

Algorithm contracts. Each solver family declares a *contract*: which input protocols it requires (with widths, capabilities, and promise objects), and which output protocols it provides. `solver.check(problem)` produces a structured report—issue codes such as INPUT_PROTOCOL, INPUT_ADAPTER, INPUT_WIDTH, INPUT_CAPABILITY, INPUT_PROMISE, OUTPUT_LAYOUT, with paths, expected and actual values, and human-readable messages—and `.require()` turns reports into exceptions. Reports serialize to JSON: contracts are machine-readable documentation, not a second executable IR.

Generator protocols. Solver families are exposed as small callables—for the ODE stack, the three-argument QODEProtocol mapping (generator, initial state, time) to a state oracle—so that *solvers are plug-compatible positions* in larger programs: a QLSS instance, an ODE solver family, or a Hamiltonian-simulation kernel can be injected where a linear solver is expected (Section 7).

```
@runtime_checkable
class HasDiagonal(Protocol):
    """An application-local protocol; the language need not know it."""
    def diagonal_values(self) -> tuple[float, ...]: ...

class GivenMatrix:
    def __init__(self):
        self.encoding = abstract_block_encoding("GivenA", 1, 0, 1.0)

    def block_encoding(self):
```

```

        return self.encoding                # open oracle slot

    def diagonal_values(self):
        return (1.0, 1.0)

a = GivenMatrix()
assert requires(a, HasDiagonal).diagonal_values() == (1, 1)
qlss = make_costa_qlss(CostaConfig(steps=1))
problem = LinearSystem(block=BlockSystem(a, gate, SpectralPromise(1,
    ↪ 1)),
                        rhs_norm=1)
report = qlss.check(problem)
report.require()                        # contract check passes
result = qlss(problem)                  # -> StateOracleProtocol
closed = bind(result.operation.program(),
              {"GivenA": identity(1).operation})

limited = abstract_state_prep("ForwardOnly", 1, reversible=False)
rejected = qlss.check(LinearSystem(
    block=BlockSystem(a, limited, SpectralPromise(1, 1))))
assert not rejected.ok                  # capability violation, reported

```

Listing 5: Protocols in action (adapted verbatim from `examples/algorithm_contracts.py`). Top: an application-defined protocol `HasDiagonal`, satisfied structurally by an ordinary class whose block-encoding slot is an open declaration. Bottom: a `Costa-QLSS` instance consumes the problem object, its contract check succeeds, the solver runs, and the resulting open program is closed by binding an implementation to the `GivenA` slot.

Promise provenance. Mathematical promises are objects with *evidence*: a spectral-promise object (`SpectralPromise`) carries norm and smallest-singular-value bounds together with a provenance string recording why the bound is believed (R6). A promise without evidence is still accepted—the language does not verify mathematics—but the declaration travels with the program and appears in contract reports, so certification work later knows exactly what was assumed. The same discipline extends to solver internals: generated modules carry attributes recording what a kernel omitted (finite quadrature plan, omitted contour poles, Carleman truncation), which Section 9 turns into an explicit per-method verification matrix.

5.4 Readout as a host-side plan

Because RIR is purely unitary (R7), measurement never blocks composition: solver outputs are state oracles that further algorithms consume coherently. When a deployment does need classical data, the host attaches *readout actions*—measure, reset, post-selection, and the statistics to compute—as a plan appended after export, referencing registers by name. Estimation primitives in the library (phase estimation, amplitude estimation with its decoder, Hadamard tests, swap tests) are themselves unitary modules whose measurement plan is standard; Section 6 describes them.

6 Primitives for quantum scientific computing

The primitive library is the layer that realizes the input-model and transform strata of Fig. 1. We describe its components in the order in which a solver consumes them: block-encoding algebra (Section 6.1), data loading and sparse access (Section 6.2), the qubitiza-

tion/QSVT/QSP engine (Section 6.3), estimation and readout primitives (Section 6.4), reversible arithmetic and the math-function compiler (Section 6.5), and the broader scientific-computing suite (Section 6.6). Table 6 summarizes.

6.1 Block-encoding algebra

A `BlockEncoding` is a first-class object: an operation together with its normalization factor α (carried as an RIR attribute) and capability flags. The library provides the closure operations that scientific-computing analyses take for granted: products, scalar rescaling, linear combinations, general LCU assembly (with square-root-weight PREPAREs), tensor products, adjoints, Kronecker sums, direct sums, projectors, truncated shift lattices (the structured encodings produced by finite-difference and stencil discretizations), Pauli words, and small explicit Pauli expansions for prototyping. Because every algebraic node is a real module graph over real oracles, α bookkeeping, capability propagation, and openness all flow through composition exactly as Section 5 prescribes (R2, R6).

6.2 Data loading and sparse access

Between a mathematical input model and a circuit lies the data-loading problem, and the library treats it as a family of interchangeable realizations behind the oracle slots of Section 5.2:

- **Select-swap QROM** with an explicit partition cost model, so that the cost of a given table shape can be read off before choosing a realization;
- **Alias-sampling PREPAREs** for LCU coefficients;
- **XOR databases** in gate, QRAM, and banked (multi-bank, wide-word) forms;
- **Sparse-access pairs**: a location oracle plus an entry XOR oracle, in gate and QRAM forms—the input model consumed by Childs–Kothari–Somma-style linear-system kernels;
- **State preparations**: gate compilations, QRAM-backed rotation-angle memories, uniform and basis states, subspace selection, and the *application* of a block-encoding to a state.

Interchangeability is testable: the example suite runs the same open problem with gate and QRAM realizations bound to the same slot and cross-checks full amplitudes on native backends—the semantics must agree bit for bit.

6.3 Qubitization, QSVT, and a self-verifying QSP engine

The transform layer is built on qubitization: the library assembles qubitization walks from PREPARE–SELECT pairs [2, 7], composes QSVT phase sequences, and applies oblivious amplitude amplification where the analysis calls for it. Its distinctive component is the *quantum signal processing phase engine*: given a target polynomial, it synthesizes QSP phases by complement-polynomial rooting (a Durand–Kerner iteration) followed by layer stripping, supports degrees up to 40, and *self-verifies* by round-tripping the synthesized phases to a residual around 10^{-9} before any circuit is built. Standard transformations are provided on top: matrix inversion (the J-polynomial response for QLSS), eigenstate filtering, Hamiltonian simulation via Jacobi–Anger cosine/sine branches, and fixed-point

search in closed form. This is currently the most numerically hardened part of the stack: it is exercised by dedicated circuit-level tests, including amplitude checks on real backends (Section 9).

6.4 Estimation and readout primitives

The estimation family is assembled from the same modularity discipline: phase estimation [42, 43] keeps its iterate powers as symbolic **Repeat** nodes (R5); amplitude estimation [44] is composed as QPE on the Grover iterate with an explicit decoder; Hadamard tests produce real and imaginary parts; swap tests compare registers. All are unitary modules whose *measurement plans* are standard host-side readouts (Section 5.4). Search infrastructure (Grover iterations, coherent amplification of zero-signal subspaces) and quantum walks (coined lattice walks and Szegedy walks with MNRS-style search, driven by pluggable neighbor-table oracles) round out the algorithmic base layer.

6.5 Reversible arithmetic and the math-function compiler

Scientific-computing circuits eat and produce *numbers*, so arithmetic is a load-bearing primitive. The library provides fixed-point reversible arithmetic—addition, subtraction, negation, absolute value, multiplication, division, reciprocal, square root, comparisons, selection, bitwise operations—as *status-flagged* networks: every path sets domain-invalid and overflow bits that propagate through composition, so a guarded reciprocal is a first-class object rather than a comment. Networks are built from an XOR/AND/NOT SSA representation with compute/uncompute scheduling over enforced-zero workspaces, and native backends may compile whole arithmetic modules to C++ operators charged unit cost (Section 4.6).

On top of arithmetic sits the *math-function compiler*, which turns ordinary pure Python functions into reversible modules without executing them:

```
from pyqecclang import FixedFormat, compile_function,
    ↪ export_toffoli_u3_cz

def pressure(rho, momentum, energy, gamma=1.4):
    velocity = momentum / rho
    return (gamma - 1) * (energy - 0.5 * momentum * velocity)

compiled = compile_function(
    pressure,
    fmt=FixedFormat(width=12, fraction=6),
    constants={"gamma": 1.4},          # generation-time parameter
)
operation = compiled.operation        # ports: rho, momentum, energy,
program = compiled.program()          # out, status
originir = export_toffoli_u3_cz(program).text
```

Listing 6: Compiling a pure Python numerical function to a reversible module (verbatim from the doctested manual; the gas-dynamics pressure function used by the QFVM pipeline of Section 8). The compiler reads the abstract syntax tree (imports restricted to math/cmath; no execution), folds constants, emits a typed SSA middle IR (MIR 0.1), and lowers to an RIR module with fixed-point ports and propagated status flags. The calling convention is the XOR-update ABI: inputs preserved, result XOR'd into the out register, domain/overflow state XOR'd into status.

The frontend performs constant folding and bounded inlining of non-recursive helpers; the middle IR is a typed SSA DAG (real/complex/bool kinds, explicit selection of booleans

into numerics) with its own schema; the lowering emits compute / XOR-update / uncompute schedules honoring the workspace ABI. Elementary kernels (exponentials, trigonometric and hyperbolic functions, square roots, two-argument arctangent) are polynomial approximations with per-intrinsic interval bounds, complex kernels decompose into real ones (e.g. $e^z = e^{\operatorname{Re} z}(\cos \operatorname{Im} z + i \sin \operatorname{Im} z)$), integer powers proceed by square-and-multiply, and domain/accuracy status flags propagate through every path. The result is that *classical numerical expertise written as Python* becomes a reversible oracle implementation—the bridge that applications such as the Roe flux (Section 8) cross.

6.6 The broader scientific-computing suite

Beyond the linear-algebra core, the library includes input models and algorithms from across quantum scientific computing, each with the same declare/carry/bind discipline: Heinrich’s quantum summation and numerical integration; Jordan’s gradient estimation from phase oracles; density-matrix input models via *purification access* (with Gibbs-state preparation by QSVT Chebyshev/Bessel truncation, partial traces, and trace distance); quantum PCA by density-matrix exponentiation; a quantum SDP solver scaffold (penalty Hamiltonians, Gibbs solves, trace estimation, and the matrix-multiplicative-weights outer loop at the host); low-rank double-factorization and tensor-hypercontraction block-encodings for quantum chemistry; and decoded quantum interferometry for GF(2) optimization. This breadth exists to exercise the language machinery across input-model variety—and it is what the ODE/PDE stack of the next section builds on.

7 Differential equations as a first-class workload

The differential-equation stack is where the three headline mechanisms compound: plugable oracles make input models declarative (Section 7.1); protocols make solver families interchangeable positions (Section 7.2); and structure preservation keeps discretized, lifted, and iterated solver skeletons small and serializable (Section 7.3). Fig. 8 maps the territory; Table 3 compares coverage against existing frameworks.

7.1 Input models: the problem layer stays native

Every ODE method consumes one of four *problem objects*, each holding its generator as open-oracle-capable objects rather than matrices:

- **QODEProblem**: $u' = Gu$ with the generator G given as a block-encoding, the initial state as a state-preparation oracle, an optional *dissipativity* promise with evidence provenance, and the initial norm;
- **LinearODE**: $u' = -Au$ with Hermitian parts $A = L + iH$ (the decomposition some methods need, declared by the caller);
- **PolynomialODE**: $u' = \sum_p F_p u^{\otimes p}$ with each F_p a block-encoding—the *nonlinear* input model, consumed by Carleman lifting;
- **FokkerPlanckProblem**: one-dimensional Itô density dynamics $p' = -\partial_x(ap) + \partial_x^2(Dp)$, discretized by zero-flux finite volumes into a generator block-encoding, with classical reference witnesses (evolution, stationary and Boltzmann distributions) attached.

Table 6: Primitive inventory, organized by the strata of Fig. 1. Every family is assembled from modules and honors the oracle lifecycle of Section 5; “open-slot ready” means the family can be generated against abstract declarations.

Family	Contents	Status
Block-encoding algebra	products, scaling, linear combinations, LCU, tensor, adjoint, Kronecker sum, direct sum, projectors, stencil shifts, Pauli words	implemented; tested; open-slot ready
Data loading	select-swap QROM (partition cost model), alias sampling, XOR databases (gate/QRAM/banked), sparse access pairs, state preparations	implemented; native cross-checked
QSP/QSVT	phase synthesis (self-verifying, degree ≤ 40), qubitization walks, oblivious amplitude amplification, inversion / eigenstate filters / Hamiltonian simulation / fixed-point search	implemented; numerically hardened
Hamiltonian simulation	first-order Pauli Trotter; truncated-Taylor LCU (default kernel); QSP kernel injectable	Trotter/Taylor implemented; general QSP kernel by injection
Estimation	phase estimation (symbolic powers), amplitude estimation + decoder, Hadamard tests, swap tests	implemented
Arithmetic & math functions	status-flagged fixed-point arithmetic; Python AST $\rightarrow$ SSA MIR $\rightarrow$ RIR compiler with polynomial kernels	implemented; native cross-checked
Scientific suite	integration (Heinrich), gradients (Jordan), density models via purification, Gibbs, QPCA, SDP scaffold, DF/THC low-rank, DQI	implemented at interface level; maturity varies (Section 9)

This is the problem–adapter–protocol discipline of Section 2 made concrete: a PDE or ODE arrives in the paradigm native to the application; adapters construct block-encodings explicitly; solvers are generator protocols. The same problem object feeds every solver family, which is precisely what makes method comparison a one-line change (Listing 7).

```

from pyqecclang import Bits, Builder, QODEProblem, identity, scale
from pyqecclang.algorithms.ode import linear_qode
from pyqecclang.algorithms.hamiltonian import taylor_hamiltonian
from functools import partial

b = Builder("initial_one", {"q": Bits(1)})
b.x(b["q"])
problem = QODEProblem(scale(-1, identity(1)), b.finish(),
                      dissipative=True, initial_norm=1.0)
kernel = partial(taylor_hamiltonian, degree=1)

lchs = linear_qode("lchs", hamiltonian_function=kernel)
schrodinger = linear_qode("schrodingerization",
    ↪ hamiltonian_function=kernel)

assert lchs.check(problem, time=0.05).ok
assert schrodinger.check(problem, time=0.05).ok
first = lchs.solve(problem, 0.05)
second = schrodinger.solve(problem, 0.05)

```

```
assert first.width == second.width == 1
```

Listing 7: One problem, two solver families (verbatim from the doctested tutorial). Both generators return one-qubit state oracles for $u' = -u$, $t = 0.05$; their ancilla structure, approximation route, and recovery conditions differ. The tutorial is explicit that this checks *assembly*: a degree-one Taylor kernel does not certify an arbitrary-precision solution—promise provenance (R6) records that boundary.

7.2 Six solver families as interchangeable protocols

LCHS (linear combination of Hamiltonian simulations). Dissipative evolution e^{Gt} is expanded as a quadrature sum of Hermitian-branch evolutions $e^{-i(H+kL)t}$ weighted by Cauchy-type coefficients; the library assembles the weighted LCU [8] with a configurable `QuadraturePlan` (default: finite Cauchy cutoff and spacing). The finite-quadrature remainder is recorded as a pending item on the generated modules—an R6 obligation, not a silent truncation.

Schrödingerization. For a general generator $G = H_1 + iH_2$, the state is lifted through a Fourier-transformed ancilla register so that evolution becomes Hamiltonian in the lifted space; the library realizes the lift with a Fourier-momentum diagonal block-encoding, warps the initial state by $e^{-|p|}$, and selects the physical channel at a planned index with an explicit recovery scale [9, 16]. Periodic-truncation caveats are recorded as pending.

Contour-based matrix-function decomposition (CBMD). Evolution (or a general matrix function) is decomposed along a contour [45] with auxiliary poles; the assembly supports arbitrary nodes and residues, and the generated modules record which auxiliary non-Hermitian evolutions and series tails were omitted.

Carleman linearization. The nonlinear path: `PolynomialODE` inputs are lifted to a cutoff- K block-triangular tensor system (the lift itself is a structured placement of trilinear blocks with level controls), the initial state becomes a weighted tensor power, any linear solver protocol then applies, and the output is projected back to the level-one subspace [12, 13]. The truncation tail is a recorded pending item.

Implicit-Euler history systems via QLSS. Time discretization first, quantum solving second: the whole solution history of the implicit-Euler scheme is assembled as one block system (an LCU of projector, shift, and generator blocks) and solved in one shot by any QLSS protocol; the final-time subspace is selected on the quantum side. In this family, *the linear-system solver is the injectable position*.

Truncated Taylor kernels. A closed polynomial candidate $\sum_k (tG)^k/k!$ with no structural assumptions—also the default *Hamiltonian-function kernel* that the three evolution-based families above inject, with degree and configuration swappable at call time.

7.3 The PDE layer: discretization, homotopy analysis, finite volumes

Three paths connect PDEs to the ODE stack, each exercising a different part of the language:

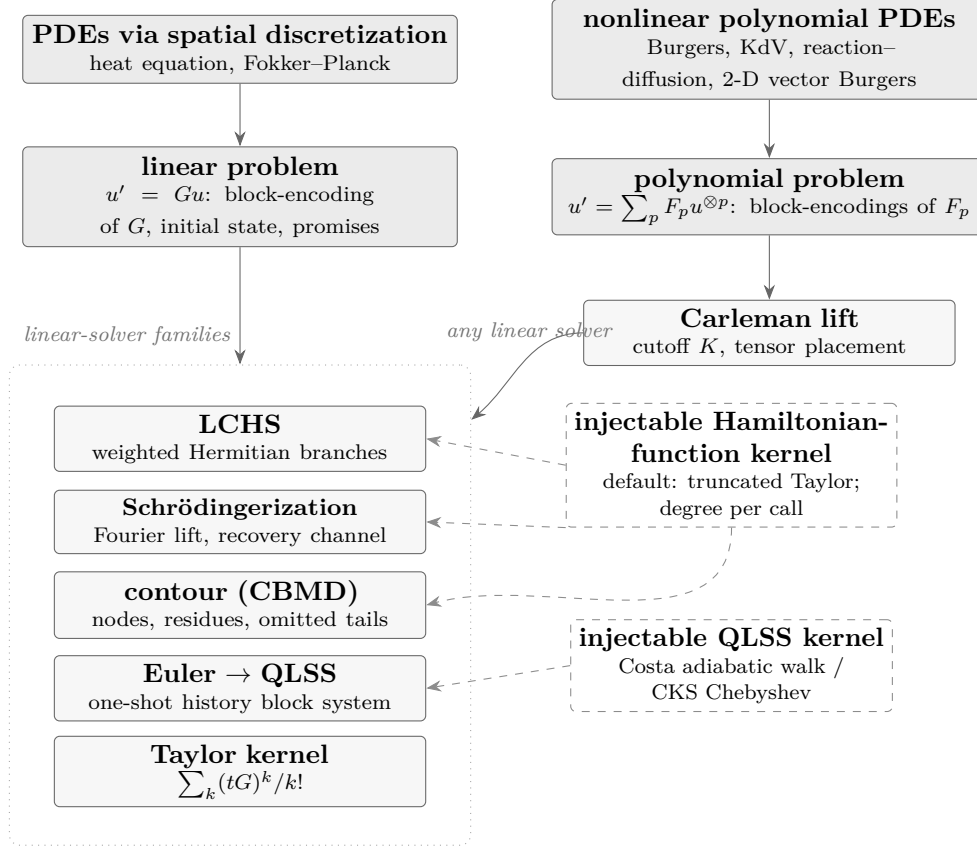


Figure 8: The ODE/PDE solver territory of pyqecclang. Linear problems feed four evolution- and system-oriented families (LCHS, Schrödingerization, contour decomposition, Taylor kernels) plus the QLSS-based history-system family; nonlinear polynomial problems lift through Carleman into the same linear-solver position. Dashed boxes mark *injectable protocols*: the kernel realizing e^{-iHt} subroutines and the linear-system kernel are swappable arguments, as are the spatial discretizers connecting the PDE layer to the problem layer.

Pluggable spatial discretization. `DiscretePDE` pairs a generator block-encoding with an initial state after spatial discretization; `qpde_solver` threads any discretizer into any QODE protocol. The built-in periodic heat equation, for instance, assembles a structured four-point shift block-encoding (the stencil shift primitive of Section 6.1) and runs under LCHS or Schrödingerization interchangeably.

QHAM: homotopy-analysis linearization of nonlinear PDEs. Symbolic polynomial PDEs—Burgers, KdV, reaction–diffusion ($u' = u - u^3$), coupled systems, and two-dimensional vector Burgers—are expressed in a *PolynomialPDE* layer; a homotopy-analysis plan (order parameter η , linearization order m) is computed *lazily* as a row-rule plan that serializes without materializing its tensor blocks; bindings then generate the lifted generator block-encoding and the norm-preserving lifted initial state; and the result is handed to any QODE solver, with `dissipative_shift` adapting it for LCHS/CBMD. Laziness here is an R5 statement: a second-order QHAM plan for a moderately sized Burgers grid is a *plan object* with a schema, not an exploded circuit.

Fokker–Planck from SDEs. Stochastic dynamics enter as Itô density evolution, are discretized by zero-flux finite volumes into Pauli-structured generators, and are solved

as `QODEProblems`; the problem object carries pure-Python classical witnesses (evolution, stationary and Boltzmann distributions) used by tests to hold the input model honest.

A note on method coverage. We are not aware of another programming system in which these solver families coexist behind a common problem-object interface with injectable linear-system and Hamiltonian-function kernels; the comparison of Table 3 makes this precise. The closest analogues are individual research codebases (one method each, no shared input model) and textbook circuit libraries (linear systems as demonstrations, no ODE/PDE stack at all). Coverage, of course, is only meaningful together with verification status, which Section 9 reports per family.

8 Case studies

Three case studies show the mechanisms compounding on real workloads: a fluid-dynamics pipeline assembled almost entirely from open oracles and compiled classical code (Section 8.1), a symbolic-PDE linearization whose intermediate artifacts are plans rather than circuits (Section 8.2), and the reproducible gallery/catalog that pins behavior across backends (Section 8.4).

8.1 QFVM: compressible Euler equations via reversible flux arithmetic

Quantum algorithms for fluid dynamics have an active literature [46, 47]; the quantum finite-volume method (QFVM) targets the one-dimensional compressible Euler equations by discretizing in a finite-volume form whose face fluxes are computed from *raw conserved fields* $(\rho, \rho v, E)$ through the frozen-Roe Riemann solver: Roe averages, an entropy fix, and the eigen-decomposition $R|\Lambda|R^{-1}$ of the flux Jacobian. Fig. 9 shows the assembled pipeline; every stage is a language mechanism in action:

- **Coherent data access.** The flow field lives in a QRAM as an XOR database of conserved variables—runtime data, locally patchable between runs, never embedded in the program (R2, R5).
- **Classical physics as compiled reversible code.** The Roe flux formulas—ordinary Python: square roots, divisions, absolute values, branching entropy fixes—are compiled by the math-function compiler (Section 6.5) into reversible fixed-point modules with propagated status flags. No matrix-entry lookup table is used: the flux is *computed* from the fields, exactly as a classical Roe solver computes it.
- **Sparse-access assembly.** The implicit step’s operator is exposed as a 9-sparse location/entry oracle pair over the conserved state, assembled through a Hermitian dilation of the rectangular flux operator plus padding identity—the sparse-access paradigm of Table 5.
- **Injector position.** The linear solve is performed by *any* QLSS protocol under an explicit **SpectralPromise**: the Costa walk kernel and the CKS Chebyshev kernel are interchangeable on the same problem object, and the contract machinery rejects inputs whose promises are missing.

The pipeline demonstrates the claim that motivated this work: nothing in it is a monolithic circuit. Each stage can be bound independently (gate tables vs. QRAM; either QLSS kernel; different fixed-point formats), and the whole is a module graph of reviewable

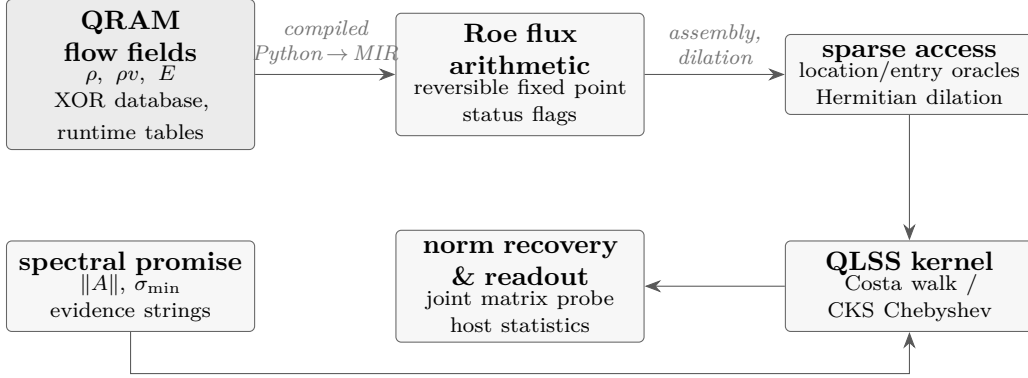


Figure 9: The QFVM pipeline for the one-dimensional compressible Euler equations. Flow fields are served coherently from a QRAM; Roe flux formulas written as pure Python are compiled to reversible arithmetic; the implicit-step operator is exposed as a sparse-access oracle pair; and the linear solve is an *injected* QLSS kernel operating under an explicit spectral promise. Its fixed-point physics is certified against an independent fixed-point semantics in the verification matrix of Section 9.

size. Its physics is certified at the fixed-point level against an independently written fixed-point semantics (Section 9.3), with generation-time constant quantization recorded as the dominant method error.

8.2 QHAM: from symbolic PDEs to solver-ready input models

The QHAM path begins at *symbolic* polynomial PDEs—equations are entered as expressions, not discretized operators—and proceeds: symbolic equations → homotopy-analysis plan (order m , parameter η) → lazy row-rule linearization plan → stencil-structured port encodings → generator block-encoding plus norm-preserving lifted initial state → any QODE solver. Two tiers of openness are available to the application: bind the structured stencil ports, or keep the lifted generator’s entire block-encoding open and carry only the plan in attributes—the open program is then a *problem specification artifact* that can be bound by whoever owns the physics (R1). The general-forced-Burgers example in the repository runs the full path at order $m = 2$, hands the result to a Schrödingerization solver, and exhibits the `dissipative_shift` adaptation used when the downstream family is LCHS or CBMD instead. Tests verify the linearization algebra itself (chain-rule identity, stencil-row equality, and reduction to the plain method at $m = 1$)—a division of labor the evidence ladder of Section 9 makes explicit.

8.3 The input-model axis

Every solver family of Section 7 varies along a second axis orthogonal to the choice of method: the *input model* through which problem data enter the program. The same mathematical instance—a forced Burgers equation for QHAM, its unforced sibling for Carleman, a given generator or a discretized heat equation for LCHS and CBMD—can be presented as structured gate stencils, as a spectral embedding (an exact Pauli-LCU decomposition of the port or generator matrices), or through QRAM-served data paths: coefficient angle databases, prepare tables, sparse-access position/entry tables, and QRAM-loaded initial states. The effect of the choice is visible in the artifacts themselves rather than in any solver code: which oracle slots remain open in the exported program, which QRAM resources appear after binding, and whether the data are exact gate truth tables or angle-quantized words at a declared precision. Appendix B.5 runs all four families

through this matrix on common instances and records the resulting open-slot and resource signatures (Table 9).

Two consequences are worth noting here. First, the QRAM variants are not special-purpose programs: the coefficient-angle database is produced by one new adapter, `qram_coefficient_encoding`, plugged into the existing `coefficient_encoder` hook of the structured finite-difference bindings, and the sparse-access grid stencil is the same **SparseA** paradigm of Section 5 promoted to a *discretization*—so a QRAM-served grid flows through `make_qpde` into any QODE family unchanged. Second, input models compose across families: the open coefficient-database QHAM input, after `dissipative_shift`, is solved by CBMD with no change to its bindings, and the same open program closes once against a gate database and once against a QRAM database with only the runtime memory table changed (R2).

8.4 The gallery and the reproducible catalog

Two artifacts anchor day-to-day verification:

- **The algorithm gallery** (22 cases) spans the base library: query algorithms, Fourier arithmetic, search and amplification, estimation (phase, amplitude, counting, Hadamard, swap tests), variational circuit generation, quantum walks, modular arithmetic and order finding, repetition codes, and Trotter evolution. Every case is exported to RIR, OriginIR-ext, and the strict gate set, and—in the native configuration—its full amplitude vector is cross-checked between the reference simulator and *both* real backends (PySparQ and the OriginIR/unique stack) to 10^{-10} . This is the circuit-witness level of the evidence ladder: it certifies that what the language generated is what the backends executed.
- **The applications catalog** (32 reproducible cases) covers the scientific-computing stack: sparse and database input models in gate *and* QRAM forms, both QLSS kernels, the ODE families, QHAM and QFVM entries, QSVT transforms, amplitude amplification, block-encoding algebra, arithmetic, batched and banked QRAM, register views, and measurement plans. Each catalog entry is an *open* program with a declared binding plan and runtime memory tables; integration tests parse every exported artifact with the real OriginIR parser.

Together they pin the meaning of “the same program” across realizations (R2) and backends, and they delimit honestly what is verified: structural validity and circuit-level agreement everywhere, and numerical certification of the scientific solvers at register-exact scale—the campaign of the next section, including the defects it exposed.

9 Implementation and verification status

This section states precisely what is implemented, what is verified, and what is not. We hold this to be part of the language’s design: because R6 requires promises and caveats to travel with programs, the same discipline applies to the paper.

9.1 Implementation

pyqecclang is a pure-Python package (version 0.8.0; Python ≥ 3.11) with zero third-party runtime dependencies. Source is layered as **infrastructure** (IR, builder, validation, serialization, math frontend, backends), **algorithms** (the primitive and solver libraries), and

applications (QFVM, QHAM mathematics, the catalog). The RIR specification (currently version 0.3), its JSON Schema, the serializer, and the semantic tests are maintained in lockstep—a norm from the project’s development conventions—and readers accept prior versions, so stored programs remain loadable across format revisions. Native backends (PySparQ; the unique OriginIR simulator) are optional environments pinned in a reviewed revision ledger rather than installed dependencies; generation and serialization work without them, and integration tests run in whichever interpreter provides them, without mocks or substitutes.

9.2 A graded evidence ladder

Verification claims are organized as levels, defined in the project’s validation plan and enforced by its test suite:

- **L0 – static:** schema validation of every serialized format; ruff-clean sources; documentation built with warnings as errors, with doctests executing every tutorial snippet quoted in this paper.
- **L1 – structural validity:** IR validation (typing, alias and control protection, call-graph acyclicity, capability demands), serialization round-trips (byte-identical, open modules included), and contract checking of algorithm inputs.
- **L2 – classical witnesses:** small-scale reference computations attached to input models (e.g. Fokker–Planck stationary distributions, QHAM linearization identities, walk hitting-time estimates) checked in tests.
- **L3 – catalog:** every catalog case exports, parses on real parsers, and satisfies its declared binding plan.
- **L4 – real backends:** full-amplitude agreement between the reference simulator and both native backends for the 22-case gallery, math-function compilations, and gate-vs-QRAM oracle realizations, to 10^{-10} ; no simulator substitutes.
- **L5 – documentation:** doctest builders over the manual.

What the ladder does *not* by itself include is numerical certification of the scientific solver stack: solution accuracy against continuum references, success-channel probabilities, and convergence in method parameters. Those are properties of algorithms, not of circuits. We therefore ran a dedicated numerical-verification campaign on the real backends, reported in Section 9.3; its outcomes are folded into the last column of Table 7, and the remaining open items are tracked per method as explicit pending items attached to generated modules (R6).

9.3 Numerical verification on real backends

We close the certification column at the scale where exact classical references exist, using a dedicated numerical campaign rather than unit tests. Two pieces of backend infrastructure were built for it and now ship with the respective simulators. First, PySparQ executes RIR natively: `pysparq.rir` parses the versioned JSON encoding and interprets it directly on sparse states, expanding the module graph (calls, **Repeat**, **Control**, **Adjoint**) at interpretation time and mapping register-aligned arithmetic onto native integer operators—so the simulator, not only the compiler, is now an independent RIR implementation. Second, UnifiedQuantum circuits expose `Circuit.to_matrix()`, the full unitary of a parsed

Table 7: Verification status by component. “Structure” = L1 validity, serialization, contract checks; “witness” = L2/L3 classical witnesses and catalog reproducibility; “native circuits” = L4 real-backend amplitude agreement of assembled circuits; “numerical certification” = solution-accuracy and success-channel checks against mathematical references in the campaign of Section 9.3 (register-exact scale; scaling studies remain open). A dash means *not yet performed*, not impossible.

Component	structure	witness	native circuits	num. cert.
RIR core, builder, serialization	✓	✓	✓	—
Oracle lifecycle (bind, capabilities)	✓	✓	✓	—
QSP phase synthesis / QSVT transforms	✓	✓	✓	✓ ^a
Data loading (QROM, QRAM, alias sampling)	✓	✓	✓	✓
Reversible arithmetic & math functions	✓	✓	✓	✓
Estimation suite (QPE, AE, Hadamard, swap)	✓	✓	✓	✓
Hamiltonian simulation (Trotter, Taylor)	✓	✓	✓	✓
QLSS kernels (Costa walk, CKS Chebyshev)	✓	✓	✓ ^b	✓
ODE families (LCHS, Schr., CBMD, Carleman, Euler–QLSS, Taylor)	✓	✓	✓ ^b	partial ^c
PDE layer (heat, Fokker–Planck, QHAM)	✓	✓	✓ ^b	✓
QFVM / Euler pipeline	✓	✓ ^d	✓ ^b	partial ^e
Scientific suite (Gibbs, QPCA, SDP, DQI, ...)	✓	✓	✓	✓

^aQSP synthesis self-verifies polynomial round-trips; transforms are certified end-to-end in Section 9.3.

^bassembled-circuit agreement on real backends. ^call six families certified against truncated-oracle references at small scale, except Schrödingerization, where the campaign exposed a momentum-sign defect that is recorded with reproductions (Section 9.3); success channels and large-scale accuracy remain pending. ^dclassical flux and flow-state witnesses. ^efixed-point physics certified against an independent fixed-point semantics at `FixedFormat(5,2)`; generation-time constant quantization dominates the method error in coarser formats.

OriginIR-ext listing including modular DEF structure, which lets us compare compiled arithmetic against classical permutation matrices elementwise. Together with the pre-existing reference executor and the event-level PySparQ adapter, every program in the campaign can be executed by up to four independently written implementations, cross-checked wherever budgets allow. Differential testing of this kind caught a genuine wraparound defect in the new interpreter (addition on a register slice wrapping at the register width instead of the view width), which is fixed and covered by regression tests on both sides.

The campaign (Table 8) comprises 473 experiments in thirteen groups, driven by `tools/run_verification.py` and archived as machine-readable JSON (`out/verification/`). Every experiment compares against a correctness oracle that is independent of the implementation—closed forms, `numpy/scipy` references, or an independently written fixed-point semantics—and most sweeps evaluate the entire input domain in a single superposed execution instead of sampling basis states. Where an algorithm combines an exact construction with a mathematical truncation (Chebyshev degrees, quadrature cutoffs, Carleman tails), *implementation error* (against the truncated oracle) and *method*

Table 8: Numerical-verification campaign: thirteen groups, 473 experiments, all passing their stated criteria on real backends. Paths: O = OriginIR-ext + UnifiedQuantum full amplitudes (incl. to_matrix); P = PySparQ RIR sparse-state execution; X = cross-checked against the reference executor and the event adapter. Headlines quote the worst-case metric over the group's instances.

Group	cases	paths	headline metrics
Reversible arithmetic & fixed point	20	O, P, X	adders permutation-exact to 64 bit; compiled polynomials ≤ 1 quantum; sin impl. 0.055 / method 0.045 (deg. 3)
Fourier & unitary transforms	39	O, P, X	QFT vs DFT 3.8×10^{-15} ($n=4$); Fourier addition 1.0×10^{-15} ; OAA defect recorded (text)
State preparation & data loading	37	O, P, X	fidelity $\geq 1 - 2 \times 10^{-16}$; QROM/select-swap $\leq 8.4 \times 10^{-17}$; alias TVD within analytic bounds
Oracle algorithms	30	O, P, X	BV recovery = 1 ($w = 3-8$, both paths); DJ/Simon exact; XOR database exhaustive 5.6×10^{-17}
Search & quantum walks	16	O, P, X	Grover curve 2.3×10^{-15} vs $\sin^2(2k+1)\theta$; MNRS vs independent reference 1.2×10^{-15}
Block encodings	73	O, P, X	diagonal/sparse/LCU blocks $\leq 1.1 \times 10^{-16}$; PySparQ BE cross-check $\leq 6.7 \times 10^{-16}$; Taylor method trend $0.141 \rightarrow 7.8 \times 10^{-4}$
Hamiltonian simulation & QSVT	27	O, P, X	Trotter order 1.008/1.013 measured (theory 1); QSVT HamSim total $\leq 4.5 \times 10^{-4}$; inversion within prescribed error
Estimation suite	41	O, P, X	QPE Dirichlet TVD $\leq 7.3 \times 10^{-16}$; QAE within the Brassard bound; Jordan gradient exact on linear oracles
ODE solvers	27	O, P, X	LCHS impl. $\leq 1.2 \times 10^{-16}$ over five input models; CBMD direction error 3.0×10^{-3} vs LCHS 4.5×10^{-2} ; Carleman tail $9.3 \times 10^{-3} \rightarrow 1.1 \times 10^{-4}$
QHAM & QFVM	15	P, X	generator matrices $\leq 1.5 \times 10^{-17}$; input-model agreement; Roe face bit-exact vs independent fixed-point semantics
Math-function gallery	28	P, X	five gallery functions + Roe face bit-exact vs independent fixed-point semantics; domain flags exact
Density, QPCA, SDP, DQI, variational, QEC	24	O, P, X	LMR trace distance with first-order scaling ratio ≈ 0.51 ; DQI planted instance TVD 4×10^{-16} ; repetition codes exact
Number theory, Heinrich, SDE, QLSS, catalog	96	O, P, X	modular multiplication $\leq 8.3 \times 10^{-17}$; order finding vs Dirichlet $\leq 8.6 \times 10^{-16}$; CKS kernel $\leq 4.9 \times 10^{-16}$; SDE Euler order 1.0001; catalog 33 cases $\leq 1.4 \times 10^{-16}$

error (of the truncation itself) are reported separately; domain flags, padding structure, and workspace cleanliness are checked branch by branch.

Three findings are worth reporting explicitly, because they are exactly the kind of evidence a language-level verification story should produce. *First, the campaign found real defects.* The oblivious-amplitude- amplification iterator assembles $RU^\dagger RU$ and realizes its documented operator to 10^{-16} , but that operator is not the amplifying sequence: the standard three-query form $URU^\dagger RU$ recovers the signal exactly on the same fixture, so the iteration is missing the closing U call. Schrödingerization assembles its advertised generator faithfully, yet the recovered signal matches the time-reversed flow to 4.8×10^{-16} under the library's positive-QFT convention; flipping the momentum sign in an independently re-assembled variant restores the correct flow to 1.3×10^{-16} . Both defects are recorded with minimal reproductions in the verification documents and are queued for repair; we report them unfixed rather than silently patched, per R6. *Second, the dominant error in the fixed-point scientific kernels is not circuit noise but generation-time constant quantization*

(e.g. $\gamma - 1 = 0.4 \mapsto 0.25$ at six-bit formats): implementation error against the compiled semantics stays at or below one quantum everywhere, and the quantization contribution is the leading term of the method error. *Third, there is a measurable backend floor:* on deeply nested LCU programs ($\sim 10^6$ expanded steps) the sparse-state backends prune junk branches below $\sim 10^{-7}$, while the reference executor and the OriginIR-ext path agree with the polynomial oracles at 10^{-15} ; all physical post-selected blocks remain consistent across paths.

The scope of this certification is deliberately register-exact: instances are small enough that amplitudes, blocks, and success probabilities can be compared essentially to machine precision, and the reported method errors characterize the mathematical truncations, not estimates of large-scale behaviour. Scaling the campaign is the subject of the next subsection.

9.4 Backend-level validation, simulation campaigns, and resource estimation

The remaining axis of validation—and in our view the decisive one for a scientific-computing language—is deliberately *reserved* for future work and summarized here only as a statement of scope:

- **Backend validation at scale.** L4 evidence currently covers small instances where full amplitudes are checkable. Large, symbolic-scale programs (big repetition counts, wide QRAM) need staged execution regimes and differential testing across the native backends; the cost-model guards and lazy expansion of Section 4.3 are the enabling machinery, but the campaigns themselves are future work.
- **Simulation campaigns for the solver stack at scale.** The campaign of Section 9.3 certifies implementations against truncated-oracle references at register-exact scale. What remains is the parameter-scaling axis—quadrature cutoffs for LCHS, recovery windows for Schrödingerization, contour plans for CBMD, Carleman cutoffs, Taylor degrees, and success channels as functions of problem size—and the repair of the two recorded defects. The promise-provenance attributes are designed to be the bookkeeping substrate for exactly these studies.
- **Resource estimation.** pyqecclang does not yet ship a resource estimator. The ingredients are in place—symbolic structure preserved end-to-end (R3/R5), QROM partition cost models, native-operator cost accounting, and per-slot realization interchangeability (R2)—in the spirit of large-scale studies such as [48], but as a language-level pass— assembling them into a first-class estimation pass (gate counts, depth, ancilla budgets, and error budgets flowing from recorded promises) is an open work item that we consider central to the language’s validation story.

We choose to leave this section as a scoped reservation rather than to report preliminary numbers: fabricating resource estimates for an uncertified solver stack would violate the very provenance discipline this paper argues for.

10 Discussion and limitations

10.1 What the design buys, and what it costs

The three mechanisms are separable in principle but reinforce each other in practice. Open oracles without structure preservation would be a typing game on top of circuits that

still explode with scale; structure preservation without protocols would preserve modules no library can assemble; protocols without provenance would check shapes while losing the mathematics. The costs are equally real. Embedding in Python adopts the host’s dynamic typing, so contract checks run at generation time rather than at a static pass; the IR’s concreteness (all widths and angles resolved) rules out symbolic parameters inside RIR, pushing parameter studies to the generation layer; and the unitary-only IR, while it keeps solver outputs composable, means hybrid quantum–classical algorithms orchestrate from the host rather than from the IR—a deliberate division that favors the fault-tolerant algorithm regime over the variational one.

10.2 Limitations of the current implementation

We enumerate the limitations most relevant to prospective users and to the future-validation program of Section 9.4:

- **Solver certification is small-scale.** All six ODE families, both QLSS kernels, QHAM, and QFVM are certified against truncated-oracle references at register-exact scale (Section 9.3), and each still carries recorded pending items (finite quadrature, omitted contour poles, Carleman tails, recovery windows, success channels); the Schrödingerization momentum-sign and oblivious-amplification iteration defects found by the campaign are recorded but not yet repaired. None is a certified production solver yet (Table 7).
- **Hamiltonian-simulation kernels are partial.** First-order Trotter and truncated Taylor are built in; a general QSP-based Hamiltonian-simulation kernel must currently be *injected* by the caller (the QSVT-based block-encoding route exists separately). Higher-order product formulas are not yet provided.
- **Amplitude-estimation variants are canonical only.** No maximum-likelihood or robust amplitude estimation; no VTAA variants for the QLSS kernels.
- **Width regime.** Single registers span at most 64 bits (a backend storage constraint); wider objects are fused multi-register views. Explicit QHAM tensor lifts are capped by this width.
- **Number theory is illustrative.** The modular-multiplication path is a bounded permutation synthesis for small registers, not scalable Shor arithmetic—and is labeled as such.
- **No resource estimator, no error budgeting**—scoped in Section 9.4.

10.3 Future work

Beyond the reserved validation axis (backend campaigns at scale, solver certification studies, and a first-class resource estimator), the natural extensions are: a general QSP Hamiltonian-simulation kernel closing the injection point; robust and maximum-likelihood amplitude estimation; a wider nonlinear-PDE repertoire on the QHAM path (systems, higher dimensions); classical-optimization protocol objects for variational workloads; and, on the formal side, a denotational semantics for RIR modules (the validation suite currently pins behavior operationally).

11 Conclusion

Quantum scientific computing has an algorithm literature that is mature, growing, and structurally consistent: problems accessed through declared mathematical input models; solutions assembled from block-encoding algebra and signal-processing transforms; precision and promises carried as first-class mathematical objects. pyqecclang is a programming language built to that shape: oracles are declared, carried open, and linked late; solver families are protocol positions into which interchangeable kernels are injected; programs remain module-level, symbolic in scale, serializable, and honest about what they assume and what they omit. Its coverage—six ODE solver families, several PDE paths, a reversible-arithmetic and math-function compiler feeding physics kernels, and a QSVT/QSP engine that verifies its own synthesis—is, to our knowledge, unmatched by existing quantum programming systems. Its verification discipline is deliberately graduated, and the campaign of Section 9.3 shows what that discipline buys: register-exact certification against independent oracles for the whole library, two real defects exposed and recorded rather than silently absorbed, and a clean boundary to what remains open—validation at scale, the defect repairs, and resource estimation, reserved rather than overclaimed. We offer the requirements analysis of Section 2 as a reusable lens: languages for the fault-tolerant scientific-computing era will be judged by how directly they let the mathematics be written—and by how honestly they track what remains unproven.

Acknowledgments

A Reproducing the examples

All listings in this paper are drawn from the repository’s doctested tutorials, examples, and tests of pyqecclang 0.8.0. The commands below regenerate the artifacts referenced in the text:

```
# tutorials and core semantics
python -m unittest discover -s tests/core -v

# gallery: 22 cases, RIR + OriginIR-ext + strict gate set exports
python examples/algorithm_gallery.py          # add --native for
    ↪ backend agreement

# oracle/protocol/QODE/QHAM demonstration programs
python examples/algorithm_contracts.py
python examples/ode_input_models.py          # 11 input-model cases
python examples/input_models.py              # 14 input-model
    ↪ variants over 4 families
python examples/general_qham.py              # forced Burgers at
    ↪ order 2
python examples/math_functions.py

# static checks
ruff check src tests examples

# numerical verification campaign (Section 9.3): 473 experiments in 13
# groups, writes machine-readable reports to out/verification/
PYTHONPATH=src <backend-python> tools/run_verification.py
```

Listing 8: Reproduction commands (generation and export have no third-party requirements; native cross-checks require the pinned backend environments).

The native configuration runs the same examples under interpreters providing `pysparq` and `uniqc`; revision pins are recorded in the repository’s backend revision ledger. The verification campaign requires both backends: the PySparQ path executes RIR natively through `pysparq.rir`, and the OriginIR-ext path uses UnifiedQuantum’s `Circuit.to_matrix()`—both contributed as part of this work.

B The four solver families in code

This appendix shows, for each of the four solver territories of Section 7, a complete input program drawn verbatim from the repository’s doctested examples (comments translated from Chinese; artifact bookkeeping is elided at the marked points). The point of interest is not the numerics but what stays *open* and what is *swappable* in each family: every listing contains at least one `abstract_*` declaration that the same program could bind differently, and every solver entry accepts its kernel configuration as an argument (R1, R2, R5).

B.1 QHAM: from a symbolic PDE to a solver-ready input model

Listing 9 is the repository’s forced-Burgers example (`examples/general_qham.py`) in full. The equation is entered *symbolically*—a `PolynomialPDE` over a `Field` expression, with the forcing term declared as named data (`Known("forcing")`) rather than a baked-in array. The homotopy plan fixes the linearization order ($m = 2$) and the homotopy parameter ($\eta = -0.4$); the structured finite-difference bindings carry the stencil-structured ports and the lifted initial state. The result, `qode_input`, is an open QODE input model: it can be handed to any solver family, and the physically owned pieces (coefficient block-encodings, forcing data) remain bindable separately.

```
from functools import partial

from pyqecclang.algorithms.hamiltonian import taylor_hamiltonian
from pyqecclang.algorithms.ode import linear_qode
from pyqecclang.applications.qham import (
    Discretization,
    Field,
    Grid,
    Known,
    PolynomialPDE,
    QHAMPlan,
    qham_input_model,
    structured_fd_bindings,
)

u = Field("u")
pde = PolynomialPDE.from_equations(
    {"u": 0.1 * u.d("x", 2) - u * u.d("x") + Known("forcing")},
    label="forced_burgers",
)

plan = QHAMPlan(pde, order=2)
grid = Grid(("x",), (4,), (1.0,), boundary="periodic")
space = Discretization(pde, grid, {"forcing": [0.05, 0, -0.05, 0]})
initial = space.encode_fields({"u": [0.1, 0.2, 0, -0.1]})
bindings = structured_fd_bindings(space, initial)
qode_input = qham_input_model(plan, bindings, eta=-0.4)

# Schrodingerization accepts a general generator; recovery-interval and
# time-approximation validation remains a pending item.
```

```

solver = linear_qode(
    "schrodingerization",
    ↪ hamiltonian_function=partial(taylor_hamiltonian, degree=1)
)
solution = qode_input.solve(solver, 0.01)

```

Listing 9: QHAM end to end: symbolic polynomial PDE, homotopy-analysis plan, structured discretization, and a solver handed in from outside (verbatim from the doctested example `examples/general_qham.py`; comments translated).

B.2 Carleman: nonlinear lift with a swappable linear solver

Listing 10 (case 6 of `examples/ode_input_models.py`) runs the inviscid Burgers equation through the Carleman pipeline. Each polynomial coefficient F_p of the lifted generator is declared as an *open* block-encoding slot (`BurgersFp`) and bound to the operation produced by the stencil ports, so the discretization could be replaced without touching the problem assembly. The lifted problem is a `PolynomialODE`; the solver, `carleman_qode(linear_solver=..., cutoff=2)`, receives the linear solver as an argument, and the loop solves the *same* nonlinear problem twice—once through Schrödingerization and once through norm-shifted LCHS—which is exactly the “Carleman lands in the same linear-solver position” claim of Section 7.2.

```

u = Field("u")
pde = PolynomialPDE.from_equations({"u": 0.1 * u.d("x", 2) - u *
    ↪ u.d("x")})
space = Discretization(pde, grid)
ports = structured_fd_bindings(space, space.encode_fields({"u": [0.1,
    ↪ 0.2, 0, -0.1]}))
concrete = polynomial_from_bindings(ports)
coefficient_slots, bindings = [], {}
for order, coefficient in concrete.coefficients:
    name = "BurgersF" + str(order)
    slot = abstract_block_encoding(
        name, coefficient.width, coefficient.signal_qubits,
        ↪ coefficient.alpha
    )
    coefficient_slots.append((order, slot))
    bindings[name] = coefficient.operation
initial_slot = abstract_state_prep(
    "BurgersInitial", concrete.width, concrete.initial.work_width
)
bindings["BurgersInitial"] = concrete.initial.operation
problem = PolynomialODE(
    concrete.width, tuple(coefficient_slots), initial_slot,
    ↪ concrete.initial_norm
)
recovery = {}
for method, solver in (
    ("schrodingerization", schrodinger),
    ("shifted_lchs", shifted_solver(lchs, recovery)),
):
    nonlinear_solver = partial(carleman_qode, linear_solver=solver,
        ↪ cutoff=2)
    state = qpde_solver(nonlinear_solver)(PDEInput(problem, "burgers"),
        ↪ time)

```

Listing 10: Carleman lift of Burgers: open coefficient block-encodings, an open lifted initial state, and a linear solver injected per call (verbatim from case 6 of `examples/ode_input_models.py`; comments translated, artifact serialization elided).

B.3 LCHS: one open program, two data-path realizations

Listing 11 condenses three cases of `examples/ode_input_models.py`. The protocol object is built by the uniform `linear_qode` front with two swappable arguments: the quadrature plan (Cauchy, cutoff 1) and the injectable Hamiltonian-function kernel. The first problem declares only an abstract generator block-encoding; the third declares a rotation-angle database, and the *same* open program is then closed twice—once against a gate realization of the database and once against a QRAM database with a runtime memory table—with no change to the program itself (R2).

```
time = 0.05
hamiltonian_function = partial(taylor_hamiltonian, degree=1)
quadrature = QuadraturePlan.cauchy(cutoff=1, spacing=1.0)
lchs = linear_qode("lchs", plan=quadrature,
    ↪ hamiltonian_function=hamiltonian_function)

# Generator block-encoding given directly;  $G = -I$ . The work qubits are
# reserved for the QRAM initial-state realization used below.
initial = abstract_state_prep("Initial", 1, work_width=3)
initial_gate = gate_state_prep([1, 1], work_width=3)
generator = abstract_block_encoding("Generator", 1, 0, 1.0)
state = lchs(generator, initial, time)
# ... artifact record elided ...

# Same open graph, bound once against a gate database and once against
# a QRAM database plus state preparation. The data are rotation-angle
# integer encodings;  $A = \text{diag}(1, \cos(\pi/4))$ ,  $G = -A$ .
angles = abstract_database("DiagonalAngles", 1, 2)
a = diagonal_block_encoding(angles, alpha=1.0)
state = lchs(scale(-1, a), initial, time)
records.append(
    write(
        "given_xor_gate_lchs",
        state,
        {
            "DiagonalAngles": gate_database(1, 2, [0, 1]).operation,
            "Initial": initial_gate.operation,
        },
    )
)
# ... and the QRAM realization of the same slots:
records.append(
    write(
        "given_xor_qram_lchs",
        state,
        {
            "DiagonalAngles": Binding(
                qram_database(1, 2).operation, {"table":
                    ↪ "diagonal_angles"}
            ),
            "Initial": Binding(qram_state_prep(1, 2).operation,
                ↪ {"angles": "initial_angles"}),
        },
    )
)
```

```

    },
    memory={
        "diagonal_angles": [0, 1],
        "initial_angles": qram_state_angles([1, 1], 2),
    },
)
)

```

Listing 11: LCHS: the protocol takes its quadrature plan and kernel as arguments; one open program is bound to a gate database or to a QRAM database with a runtime table (verbatim from `examples/ode_input_models.py`, inside `main()`; comments translated, artifact records elided where marked).

B.4 CBMD: the contour family under a dissipative shift

The CBMD family is reached through the same uniform front—the only difference from Listing 7 is the family name string. Because the contour construction requires a strictly dissipative generator, the QHAM input model of Listing 9 first applies its `dissipative_shift` adaptation (the explicit global shift $G \mapsto G - \mu I$ with the host tracking the norm bookkeeping); the shifted model then solves against "cbmd" with the default contour configuration and the same injectable kernel. Listing 12 is the final block of `examples/general_qham.py`: two lines separate the Schrödingerization solution of Listing 9 from the CBMD solution of the same problem.

```

# For CBMD/LCHS, apply the explicit global shift that makes the
# generator dissipative first.
shifted = qode_input.dissipative_shift()
cbmd_solution = shifted.solve(
    linear_qode("cbmd",
        ↪ hamiltonian_function=partial(taylor_hamiltonian, degree=1)),
    ↪ 0.01
)

```

Listing 12: CBMD: the same QHAM input model after `dissipative_shift`, solved by naming the family (verbatim from the final block of `examples/general_qham.py`; comments translated).

Across the four families the pattern is constant: symbolic or structured problem entry, oracle slots declared open at the point where physics or hardware ownership is contested, and solver families selected by name with their approximation kernels injected as ordinary arguments. None of the listings constructs a circuit; the closest any of them comes to circuit vocabulary is the binding dictionary that names which realization closes which slot.

B.5 One problem, many data paths: the input-model matrix

Sections B.1–B.4 fixed one input model per family; `examples/input_models.py` instead holds the *problem* fixed and varies the data path, producing the fourteen artifacts of Table 9. Three input-model families appear. *Structured gate stencils* bake the stencil and coefficient data into gate logic (the baseline of the previous subsections). *Spectral embeddings* replace the structured construction by an exact Pauli-LCU decomposition of each port matrix (`gate_bindings`) or of a given generator (`matrix_pauli_encoding`); the same decomposition with a `qram_prepare` table turns the Pauli coefficients into QRAM runtime data. *QRAM data paths* leave the data in named resources: coefficient values become angle words in an open database slot (`qram_coefficient_encoding`, plugged into

Table 9: The input-model matrix of `examples/input_models.py`. All artifacts share the same small instances (forced Burgers for QHAM, Burgers with a space-dependent viscosity for the Carleman QRAM row, a given 2×2 generator, and a four-node ring Laplacian). “Open slots” lists what remains unbound in the exported program; “runtime tables” lists the QRAM resources present after closing (memory supplied separately). Gate rows are exact; angle-table rows carry the declared `angle_width` quantization.

Family	Input model	Open slots	Runtime tables
QHAM	structured stencil ports (gate)	—	—
QHAM	spectral ports (Pauli LCU)	—	—
QHAM	coefficient-angle db + declared initial	coefficient db, initial	<code>coeff_angles</code> , <code>initial_angles</code> (gate table or QRAM)
Carleman	structured stencil ports	F_1, F_2 , initial	—
Carleman	spectral ports (Pauli LCU)	F_1, F_2 , initial	—
Carleman	viscosity field via angle database + QRAM initial	ν db, F_1, F_2 , initial	<code>nu_angles</code> , <code>initial_angles</code>
LCHS	given spectral generator (gate)	Generator, Initial	—
LCHS	given spectral generator, QRAM PREPARE	Generator, Initial	<code>spectral_angles</code>
LCHS	QRAM grid stencil + QRAM initial	position, entry, initial	<code>positions</code> , <code>inverse_positions</code> , <code>entries</code> , <code>grid_angles</code>
CBMD	given spectral generator (gate)	Generator, Initial	—
CBMD	given spectral generator, QRAM PREPARE	Generator, Initial	<code>spectral_angles</code>
CBMD	QRAM grid stencil + QRAM initial	position, entry, initial	<code>positions</code> , <code>inverse_positions</code> , <code>entries</code> , <code>grid_angles</code>
CBMD	QHAM coefficient input after <code>dissipative_shift</code>	coefficient db, initial	<code>coeff_angles</code> , <code>initial_angles</code>

the `coefficient_encoder` hook of the structured bindings), the grid’s sparse-access position and entry oracles become QRAM tables, and the initial state is loaded through `qram_state_prep`.

Listing 13 shows the QHAM row: the coefficient database and the initial state are declared open, the program is solved once, and the *same* open program closes against a gate truth table or against a QRAM database with a runtime table. Listing 14 shows the grid row: the four-node ring Laplacian is exposed as a sparse-access oracle pair whose position and entry tables are QRAM resources, wrapped as a `DiscretePDE` and solved unchanged by two different families.

```

profile = [0.1, 0.2, 0.15, 0.05] # QRAM angle tables take non-negative
    ↪ amplitudes
qinitial = space.encode_fields({"u": profile})
encoder = partial(qram_coefficient_encoding, angle_width=8)
open_bindings = structured_fd_bindings(space, qinitial,
    ↪ coefficient_encoder=encoder)
open_bindings = QHAMBindings(
    open_bindings.state_width,
    open_bindings.ports,
    abstract_state_prep("QhamInitial", open_bindings.state_width, 10),
    open_bindings.initial_norm,
)
model = qham_input_model(plan, open_bindings, eta=-0.4)
state = model.solve(schrodinger, 0.01)

```

```

# ... the coefficient database slot is discovered from the open program
↪ ...
records.append(write("qham_coeff_gate", state, {
    db_slot: gate_database(2, 8, word_table).operation,
    "QhamInitial": gate_state_prep(profile, work_width=10).operation,
}))
records.append(write("qham_coeff_qram", state, {
    db_slot: Binding(qram_database(2, 8).operation, {"table":
        ↪ "coeff_angles"}),
    "QhamInitial": Binding(qram_state_prep(2, 8).operation, {"angles":
        ↪ "initial_angles"}),
}, memory={
    "coeff_angles": word_table,
    "initial_angles": qram_state_angles(profile, 8),
}))

```

Listing 13: QHAM with a coefficient-angle database: one open program, closed against a gate database or a QRAM database (condensed from `examples/input_models.py`; comments translated, slot discovery and artifact records elided).

```

fmt = FixedFormat(4, 1)
access = abstract_sparse_access("RingStencil", 2, fmt.width, sparsity=3)
laplacian = real_symmetric_sparse_encoding(access, fmt, 2.0,
    ↪ diagonal_nonnegative=True)
heat = DiscretePDE(
    scale(-0.1, laplacian),
    abstract_state_prep("GridInitial", 2, 10),
    label="ring_heat",
)
# ... forward/inverse position tables, entry table, angle table elided
↪ ...
for family, solver in (("lchs", lchs), ("cbmd", cbmd)):
    state = make_qpde(solver)(heat, time)

```

Listing 14: A QRAM-served grid discretization: the ring Laplacian as a sparse-access oracle pair with QRAM position and entry tables, plus a QRAM initial state; `make_qpde` threads it into two solver families unchanged (verbatim from `examples/input_models.py`; comments translated, runtime tables elided).

The matrix makes the language-level point of Section 8.3 concrete: input models are a property of the *program*, not of the solver. Nothing in the LCHS, CBMD, Schrödingerization, or Carleman entries changes as the data path moves from gate tables to QRAM resources; the difference is entirely carried by which slots stay open, which resources the closed program declares, and which runtime tables the host must supply. The limitations of the angle-table realizations are recorded honestly in the artifacts: QRAM state tables accept non-negative real amplitudes, and angles are quantized to `angle_width` bits. Note that the gate and QRAM closures of the same program encode *identical* data—the gate truth table stores the very angle words the QRAM table would—so the two realizations agree exactly, and the declared quantization is the only approximation relative to the exact coefficients.

C Glossary of recurring identifiers

The code surface on which this paper relies is deliberately small—fifteen names cover every program printed here, from the minimal Bell listing to the QFVM pipeline. Two

conventions organize them. First, the layering of Fig. 1 is visible in the prefixes: **Builder** and **Operation** belong to the construction layer, unadorned names such as **Module**, **Ref**, and **Load** are RIR concepts, and the **abstract_*** family lives in the algorithm libraries, which declare what a solver *needs* without constructing anything. Second, solver entry points are uniform fronts rather than class hierarchies: **linear_qode** and **carleman_qode** take the family as data (a name string) together with injectable protocol arguments, which is what allows the one-line family swaps shown in Listing 7 and Appendix B.

Table 10: Recurring identifiers used throughout the paper.

Identifier	Meaning
Builder	embedded module constructor over typed register views
Operation	finished builder product; satisfies unitary / state-preparation / block-encoding protocols
Module (RIR)	named typed interface + body (or absent body = open oracle slot)
Ref / Span	declarative register view; (register, start, width)
QRAM	typed resource: address width, data width, runtime table binding
Load	reversible memory access $ a\rangle d\rangle \mapsto a\rangle d \oplus M[a]\rangle$
bind / unresolved	oracle linking pass / open-slot gap analysis
BlockEncoding	operation + normalization α + capabilities
QODEProblem	linear ODE input object (generator BE, initial state, promises)
linear_qode	uniform entry over LCHS / Schrödingerization / CBMD with injectable Hamiltonian-function kernel
carleman_qode	nonlinear path: polynomial ODE $\rightarrow$ cutoff- K lift $\rightarrow$ any linear solver
qram_coefficient_encoding	coefficient diagonal as an open angle database, bound to a gate or QRAM realization
make_costa_qlss / make_cks_qlss	the two QLSS kernels (interchangeable)
SpectralPromise	norm / smallest-singular-value bounds with provenance
compile_function	pure-Python function $\rightarrow$ SSA MIR $\rightarrow$ reversible RIR module
quantikz	circuit-diagram export; calls as named boxes, open slots dashed
export_originir	modular textual export (module calls preserved)

The table doubles as a stability statement: these names constitute the public surface of pyqcclang 0.8.0 on which every doctest, gallery case, and catalog entry of Appendix A is pinned. Readers who want the next level of detail—constructor signatures, protocol definitions, and the per-family configuration records—will find them in the API reference shipped with the distribution, which is generated from the same docstrings the examples exercise.

References

- [1] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. “Quantum algorithm for linear systems of equations”. *Physical Review Letters* **103**, 150502 (2009).
- [2] András Gilyén, Yuan Su, Guang Hao Low, and Nathan Wiebe. “Quantum singular value transformation and beyond: Exponential improvements for quantum hamiltonian simulation”. In Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC 2019). *Pages 193–204*. (2019).
- [3] Guang Hao Low and Isaac L. Chuang. “Optimal hamiltonian simulation by quantum signal processing”. *Physical Review Letters* **118**, 010501 (2017).

- [4] John M. Martyn, Zane M. Rossi, Andrew K. Tan, and Isaac L. Chuang. “Grand unification of quantum algorithms”. *PRX Quantum* **2**, 040203 (2021).
- [5] Dominic W. Berry, Andrew M. Childs, Richard Cleve, Robin Kothari, and Rolando D. Somma. “Simulating hamiltonian dynamics with a truncated Taylor series”. *Physical Review Letters* **114**, 090502 (2015).
- [6] Andrew M. Childs, Yuan Su, Minh C. Tran, Nathan Wiebe, and Shuchen Zhu. “Theory of Trotter error with commutator scaling”. *Physical Review X* **11**, 011020 (2021).
- [7] Guang Hao Low and Isaac L. Chuang. “Hamiltonian simulation by qubitization”. *Quantum* **3**, 163 (2019).
- [8] Dong An, Andrew M. Childs, Lin Lin, and Lexing Ying. “Laplace transform-based quantum eigenvalue transformation via linear combination of Hamiltonian simulation”. *SIAM Journal on Computing* **55**, 376–409 (2026).
- [9] Shi Jin, Nana Liu, and Yue Yu. “Quantum simulation of partial differential equations via Schrödingerization”. *Physical Review Letters* **133**, 230602 (2024).
- [10] Sarah K. Leyton and T. J. Osborne. “A quantum algorithm to solve nonlinear differential equations” (2008). [arXiv:0812.4423](#).
- [11] Dominic W. Berry. “High-order quantum algorithm for solving linear differential equations”. *Journal of Physics A: Mathematical and Theoretical* **47**, 105301 (2014).
- [12] Jin-Peng Liu, Herman Øie Kolden, Hari K. Krovi, Nuno F. Loureiro, Konstantina Trivisa, and Andrew M. Childs. “Efficient quantum algorithm for dissipative nonlinear differential equations”. *Proceedings of the National Academy of Sciences* **118**, e2026805118 (2021).
- [13] Jin-Peng Liu, Dong An, Di Fang, Jiasu Wang, Guang Hao Low, and Stephen Jordan. “Efficient quantum algorithm for nonlinear reaction–diffusion equations and energy estimation”. *Communications in Mathematical Physics* **404**, 963–1020 (2023).
- [14] Ashley Montanaro and Sam Pallister. “Quantum algorithms and the finite element method”. *Physical Review A* **93**, 032324 (2016).
- [15] Andrew M. Childs and Jin-Peng Liu. “Quantum spectral methods for differential equations”. *Communications in Mathematical Physics* **375**, 1427–1457 (2020).
- [16] Weizhi Hu, Shi Jin, Nana Liu, and Lei Zhang. “Quantum circuits for partial differential equations via Schrödingerisation”. *Quantum* **8**, 1563 (2024).
- [17] Vittorio Giovannetti, Seth Lloyd, and Lorenzo Maccone. “Quantum random access memory”. *Physical Review Letters* **100**, 160501 (2008).
- [18] Danial Dervovic, Mark Herbster, Peter Mountney, Simone Severini, Naïri Usher, and Leonard Wossnig. “Quantum linear systems algorithms: A primer” (2018). [arXiv:1802.08227](#).
- [19] Paul Hudak. “Building domain-specific embedded languages”. *ACM Computing Surveys* **28**, 196 (1996).
- [20] Xiao Liu and John Kubitowicz. “Chisel-Q: Designing quantum circuits with a scala embedded language”. In 2013 IEEE 31st International Conference on Computer Design (ICCD 2013). *Pages* 427–434. (2013).
- [21] Pedro C. S. Costa, Dong An, Yuval R. Sanders, Yuan Su, Ryan Babbush, and Dominic W. Berry. “Optimal scaling quantum linear-systems solver via discrete adiabatic theorem”. *PRX Quantum* **3**, 040303 (2022).
- [22] Peter Selinger. “Towards a quantum programming language”. *Mathematical Structures in Computer Science* **14**, 527–586 (2004).
- [23] Bettina Heim, Mathias Soeken, Sarah Marshall, Christopher Granade, Martin Roetteler, Alan Geller, Matthias Troyer, and Krysta Svore. “Quantum programming languages”. *Nature Reviews Physics* **2**, 709–722 (2020).

- [24] Shruti Garhwal, Mohamad Ghorani, and Amir Ahmad. “Quantum programming language: A systematic review of research topic and top cited languages”. *Archives of Computational Methods in Engineering* **28**, 289–310 (2021).
- [25] Alexander S. Green, Peter LeFanu Lumsdaine, Neil J. Ross, Peter Selinger, and Benoît Valiron. “Quipper: A scalable quantum programming language”. In Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2013). *Pages 333–342*. (2013).
- [26] Ali JavadiAbhari, Shruti Patil, Daniel Kudrow, Jeff Heckey, Alexey Lvov, Frederic T. Chong, and Margaret Martonosi. “ScaffCC: Scalable compilation and analysis of quantum programs”. *Parallel Computing* **45**, 2–17 (2015).
- [27] Artur Scherer, Benoît Valiron, Siun-Chuon Mau, Scott Alexander, Eric van den Berg, and Thomas E. Chapuran. “Concrete resource analysis of the quantum linear-system algorithm used to compute the electromagnetic scattering cross section of a 2D target”. *Quantum Information Processing* **16**, 60 (2017).
- [28] François Fillion-Gourdeau, Steve MacLean, and Raymond Laflamme. “Algorithm for the solution of the Dirac equation on digital quantum computers”. *Physical Review A* **95**, 042343 (2017).
- [29] Jennifer Paykin, Robert Rand, and Steve Zdancewic. “QWIRE: A core language for quantum circuits”. In Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2017). *Pages 846–858*. (2017).
- [30] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. “Proto-quipper with dynamic lifting”. *Proceedings of the ACM on Programming Languages* **7**, 309–334 (2023).
- [31] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. “Silq: A high-level quantum language with safe uncomputation and intuitive semantics”. In Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2020). *Pages 286–300*. (2020).
- [32] Damian S. Steiger, Thomas Häner, and Matthias Troyer. “ProjectQ: An open source software framework for quantum computing”. *Quantum* **2**, 49 (2018).
- [33] Robert S. Smith, Michael J. Curtis, and William J. Zeng. “A practical quantum instruction set architecture” (2016). [arXiv:1608.03355](https://arxiv.org/abs/1608.03355).
- [34] Cirq Developers. “Cirq”. Zenodo (2020). Version 0.8.0; <https://github.com/quantumlib/Cirq>.
- [35] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. “Quantum computing with Qiskit” (2024). [arXiv:2405.08810](https://arxiv.org/abs/2405.08810).
- [36] Andrew W. Cross, Ali Javadi-Abhari, Thomas Alexander, Niel de Beaudrap, Lev S. Bishop, Steven Heidel, Colm A. Ryan, Prasahnt Sivarajah, John Smolin, Jay M. Gambetta, and Blake R. Johnson. “OpenQASM 3: A broader and deeper quantum assembly language”. *ACM Transactions on Quantum Computing* **3**, 1–50 (2022).
- [37] Raphael Seidel, Nikolay Tcholchev, Sebastian Bock, and Manfred Hauswirth. “Uncomputation in the Qrisp high-level quantum programming framework”. In Reversible Computation (RC 2023), Lecture Notes in Computer Science. *Pages 150–165*. (2023).
- [38] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. “t|ket>: A retargetable compiler for NISQ devices”. *Quantum Science and Technology* **6**, 014003 (2021).
- [39] Matthew Amy and Vlad Gheorghiu. “staq—a full-stack quantum processing toolkit”. *Quantum Science and Technology* **5**, 034016 (2020).

- [40] Krysta M. Svore, Alan Geller, Matthias Troyer, John Azariah, Christopher Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, Andres Paz, and Martin Roetteler. “Q#: Enabling scalable quantum computing and development with a high-level DSL”. In *Proceedings of the Real World Domain Specific Languages Workshop 2018 (RWDSL 2018)*. Pages 1:1–1:10. (2018).
- [41] Alexander M. Dalzell, András Gilyén, Connor T. Hann, Sam McArdle, Grant Salton, Quynh T. Nguyen, Aleksander Kubica, and Fernando G. S. L. Brandão. “A distillation-teleportation protocol for fault-tolerant QRAM” (2025). [arXiv:2505.20265](#).
- [42] A. Yu. Kitaev. “Quantum measurements and the Abelian stabilizer problem” (1995). [arXiv:quant-ph/9511026](#).
- [43] Michael A. Nielsen and Isaac L. Chuang. “Quantum computation and quantum information”. [Cambridge University Press](#). (2000).
- [44] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp. “Quantum amplitude amplification and estimation”. *Contemporary Mathematics* **305**, 53–74 (2002).
- [45] Chao Wang. “Quantum simulation of non-Hermitian special functions and dynamics via contour-based matrix decomposition”. *Quantum Science and Technology* **11**, 035027 (2026). [arXiv:2511.10267](#).
- [46] Zhaoyuan Meng and Yue Yang. “Quantum computing of fluid dynamics using the hydrodynamic Schrödinger equation”. *Physical Review Research* **5**, 033182 (2023).
- [47] Satish S. Bharadwaj and Katepalli R. Sreenivasan. “Towards simulating fluid flows with quantum computing”. *Sādhanā* **50**, 57 (2025).
- [48] Craig Gidney and Martin Ekerå. “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits”. *Quantum* **5**, 433 (2021).